De l'Épée au Sabre for Speedier Ripostes

Adithya Vadapalli
Kyle Storrier
Ryan Henry

ABSTRACT

We present Sabre, a sender-anonymous messaging system with instances operating in both the sender-anonymous bulletin board and sender-anonymous mailbox models. Sabre provides strong resistance against traffic-analysis attacks while potentially scaling to anonymity sets in the *billions* of users. In the sender-anonymous bulletin board mode, it improves on the celebrated Riposte scheme by replacing its comparatively sluggish and heavyweight *sqrt*-sized distributed point functions (DPFs) with decidedly nimbler and lighter-weight logarithmic-sized ones. In the sender-anonymous mailbox model, it improves on the Riposte successor Express by introducing asymptotically faster audit protocols. Compared with both Riposte and Express, Sabre provides speedups that exceed an order of magnitude when all the write requests are "well-formed" and increase significantly—with the performance gap growing exponentially—under DoS attacks.

1 INTRODUCTION

The Internet was supposed to be a great equalizer; instead, it has grown to be a great surveillance apparatus. Today, netizens are routinely identified, tracked, profiled, censored, and silenced by countless shadowy figures, organizations, governments, identity thieves—and even algorithms. Far from the bastion of free speech espoused by the web's early pioneers, the modern Internet is, in many respects, dystopic to extremes that would make even Orwell blush.

The causes are systemic: the core Internet protocols were, quite simply, not designed with privacy in mind. This apathetic approach to privacy permeates through to the application layer, culminating in today's burgeoning Internet marketplace in which consumers are the product. If we are to have any hope of reneging on this Faustian bargain, it will be through the development and deployment of protocols that respect the privacy and autonomy of all users.

One tool that can help in this regard is *sender-anonymous messaging* (SAM) protocols, including celebrated recent works like Pung [2], Riposte [9], and Express [11]. Such protocols can form the backend for, among other things, whistleblower drop boxes, anonymous email systems, or *Twitter*-like broadcast media with strong and provable privacy guarantees. This paper proposes a novel SAM protocol that subsumes and fortifies both Riposte and Express, supporting the same set of functionalities with lower overhead and superior resistance to denial-of-service attacks.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
CCS'21, November 14-19, 2021, Seoul, South Korea
© 2021 Copyright held by the owner/author(s).
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM.
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1.1 Sender-anonymous messaging

Prior work in this space has considered one of two primary settings for SAM protocols: (i) the *sender-anonymous bulletin-board* model and (ii) the *sender-anonymous mailbox* model. Protocols that operate in this first model—exemplified by Riposte [9]—support *Twitter*-like broadcast messaging while severing the link between authors and their tweets. Protocols that operate in the second model—exemplified by Express [11]—support *Secure Drop*–like mailboxes into which whistleblowers can leak documents and tips anonymously to journalists, law enforcement, and watchdogs.

Sender-anonymous bulletin boards. In the sender-anonymous bulletin board model, two or more semi-honest and non-colluding servers jointly host a shared, sparse database (the "bulletin board") comprising a large number of write locations called *buckets*. Senders can write arbitrary messages into arbitrary buckets without disclosing to the servers (or the subsequent readers) which particular messages they penned and which were penned by others. For this to work, senders must always write their messages to *uniform random* buckets—potentially clobbering messages previously written to those buckets by other senders. Therefore, the number of buckets must be large enough that the probability of accidental collisions is low, implying that the bulletin board requires $O(m^2)$ buckets to accommodate m written messages *sans* any expected collisions. Readers obtain messages of interest from the bulletin board either "in the clear" (an undeniably leaky proposition) or via some oblivious means such as private information retrieval (PIR) [7].

Sender-anonymous mailboxes. In the sender-anonymous mailbox model, two or more semi-honest and non-colluding servers jointly host a collection of registered "mailboxes". Senders can deposit arbitrary documents into specific mailboxes if and only if they know the corresponding *mailbox addresses*—cryptographically long, unpredictable bit strings—without disclosing to the servers or mailbox owners which particular documents they deposit into which particular mailboxes. The use of registered mailboxes obviates the need both for PIR

and for collision-avoidance and -recovery strategies of the sort required in the sender-anonymous bulletin board model, potentially enabling constructions that are significantly more performant and scalable. However, it comes at the cost of limiting the use cases to single-recipient applications like email and secure drop boxes.

1.2 Denial-of-service attacks

SAM constructions, both in the sender-anonymous bulletin board model and in the sender-anonymous mailbox model, may be susceptible to *denial of service* (DoS) attacks, wherein a hostile entity seeks to prevent others from distributing messages via the service. Such an attacker can employ at least two distinct tactics: (i) submit malformed write requests in a bid to corrupt the entire database, or (ii) unleash an army of Sybil's masquerading as legitimate senders in a bid to overwhelm the servers' resources.

Effective defences against the first style of DoS attacker represents the primary technical innovation of prior work on SAM protocols; the latter style of DoS attacker, by contrast, has received scant mention in the literature. At first blush, this might seem a reasonable state of affairs, as virtually all Internet-connected systems are (unavoidably) susceptible to *some* flavour of resource exhaustion–style DoS attacks. Nevertheless, we contend that the threat of such attacks is especially acute in SAM protocols, where they can serve as indirect means for silencing or deanonymizing would-be bulletin board posters and whistleblowers attempting to leak documents to a secure drop box. Looking ahead, we observe that Express, the state-of-the-art SAM protocol for the sender-anonymous mailbox model, actually grants would-be resource exhaustion–style DoS attackers an *exponential* amplification factor: with just $O(\lg n)$ ultralightweight operations, a DoS attacker can force the servers to perform $O(n)$ comparatively expensive operations. (By comparison, even our most secure Sabre construction in the sender-anonymous mailbox model requires each server to do a small constant factor *less* work than the DoS attacker.)

1.3 Overview of the state of the art

We now provide a brief overview of the state-of-the-art SAM constructions in the sender-anonymous bulletin board and mailbox models. Both constructions leverage so-called *distributed point functions* (DPFs) to implement oblivious writes, employing probabilistic *audit protocols* based on *secure multiparty computation* (MPC) or *zero-knowledge proofs* (ZKPs) to weed out malformed DPFs that would otherwise corrupt the database. We give an in-depth treatment of DPFs in Section 2.1; for now, it suffices to regard a DPF as a compact, functional representation of a secret-shared *1-hot* vector in which the sole non-zero component encodes both the message to write and the location (i.e., bucket or mailbox) at which to write it.

1.3.1 Riposte. The state-of-the-art system in the sender-anonymous bulletin board model is the Riposte scheme of Corrigan-Gibbs, Boneh, and Mazières [9]. Riposte uses a simple $O(\sqrt{n})$-sized DPF to implement oblivious writes into a bulletin board consisting of n buckets. To obtain good concrete communication and computation costs, the authors propose using a bulletin board with just $n = 19.5m$ buckets (in contrast to the $O(m^2)$ buckets dictated by the birthday bound) so that *any given message* (out of m) will collide with another message for around 5% of the writes.

Riposte's audit protocol comes in two variants: (i) a lightweight, 3-server variant based on *probabilistic batch testing* and *secure (2+1)-party computation*, and (ii) a comparatively heavyweight ℓ-server variant based on so-called *seed-homomorphic pseudorandom generators* and *zero-knowledge proofs of knowledge* (ZKPoKs). Because Corrigan-Gibbs et al.'s manuscript does not present details of—nor does their codebase implement—the ℓ-server ZKPoKs, we describe here (and later compare Sabre against) only the lighter-weight 3-server Riposte variant.

Consider a 3-server Riposte instance with security parameter $\lambda \in \mathbb{N}$ (say, $\lambda = 128$) whose bulletin board comprises $n = 19.5m$ buckets each capable of holding a single L-bit message and, for ease of exposition, suppose that $n = hw$ with $\lambda h \approx Lw$. Further, let $G\colon \mathbb{F}_{2^\lambda} \to (\mathbb{F}_{2^L})^{1\times w}$ be a cryptographically secure *pseudorandom generator* (PRG). To post a message $M \in \mathbb{F}_{2^L}$ to the bulletin board, the sender

(1) samples a random bucket index $i \in_{\mathrm{R}} [0..n-1]$ and then rewrites it as $i = qw + r$ with $0 \le q < h$ and $0 \le r < w$;
(2) samples two column vectors $\vec{s}_0, \vec{s}_1 \in_{\mathrm{R}} (\mathbb{F}_{2^\lambda})^{h\times 1}$ having a 1-hot sum $\vec{s}_0 + \vec{s}_1$ whose sole non-zero element resides in the qth row and has a least-significant bit of 1;
(3) computes $\vec{v} := G(\vec{s}_0[q]) + G(\vec{s}_1[q]) + (M\vec{e}_r)$, where $\vec{s}_0[q]$ and $\vec{s}_1[q]$ respectively denote the qth rows of $\vec{s}_0$ and $\vec{s}_1$, and where $\vec{e}_r \in (\mathbb{F}_{2^L})^{1\times w}$ is the standard basis vector having a 1 in its rth column; and then, finally,
(4) sends $(\vec{s}_b, \vec{v})$ to server b for $b = 0, 1$.

Upon receiving $(\vec{s}_b, \vec{v})$, server b forms the matrix

$$M_b := \begin{pmatrix} G(\vec{s}_b[0]) \\ G(\vec{s}_b[1]) \\ \vdots \\ G(\vec{s}_b[h-1]) \end{pmatrix} + \vec{v} \cdot \begin{pmatrix} \mathrm{lsb}(\vec{s}_b[0]) \\ \mathrm{lsb}(\vec{s}_b[1]) \\ \vdots \\ \mathrm{lsb}(\vec{s}_b[h-1]) \end{pmatrix} \in (\mathbb{F}_{2^L})^{h\times w},$$

which it adds component-wise to the bulletin board to effectuate the write.

An easy calculation verifies that $M_0 + M_1 \in (\mathbb{F}_{2^L})^{h\times w}$ has message M in its (q, r)th component and zero elsewhere—*provided the sender generated the* $(\vec{s}_b, \vec{v})$ *honestly*, which the servers confirm via the audit protocol. In the audit protocol, server b sums the rows of M_b to get $\vec{m}_b \in (\mathbb{F}_{2^L})^{1\times w}$, and then all three servers run a secure $(2+1)$-party computation to

verify that $\vec{m}_0 + \vec{m}_1$ is indeed 1-hot. Notice that auditing requires $O(nL)$ work by each of server 0 and server 1.

Express. The state-of-the-art system in the sender-anonymous mailbox model is the Express scheme of Eskandarian, Corrigan-Gibbs, Zaharia, and Boneh [11]. Express uses $O(\lg n)$-sized DPFs of Boyle, Gilboa, and Ishai [3] to implement oblivious writes into pre-registered mailboxes. Specifically, a recipient (perhaps anonymously) registers a mailbox with the servers and receives a uniform λ-bit string *addr* (the "*mailbox address*"). A sender can deposit a message into the mailbox if and only if it knows *addr*. If the registered mailboxes form a sufficiently sparse subset of $\{0,1\}^\lambda$, then attempts to guess a valid address fail with overwhelming probability.

Express requires (and supports) just two servers. Consider an Express instance with security parameter $\lambda \in \mathbb{N}$ (say, $\lambda = 128$) having n registered mailboxes addressed by $addr_1, \ldots, addr_n$. To deposit a message $M \in \mathbb{F}_{2^L}$ into the mailbox addressed by $addr_i$, the sender samples a DPF representation $(\mathsf{dpf}_0, \mathsf{dpf}_1)$ of $M\vec{e}_{addr_i}$, where $\vec{e}_{addr_i} \in (\mathbb{F}_{2^L})^{1\times 2^\lambda}$ is the standard basis vector having a 1 in its $(addr_i)$th column, and then it sends dpf_b to server b for $b = 0, 1$.

Upon receiving dpf_b, server b expands it to the vector $M_b \in (\mathbb{F}_{2^L})^{1\times n}$ in which the jth component is obtained by evaluating dpf_b at $addr_j$ for $j = 1, \ldots, n$; server b adds M_b to its mailbox database to effectuate the write.

An easy calculation verifies that $M_0 + M_1 \in (\mathbb{F}_{2^L})^{1\times n}$ has message M in its ith column and zero elsewhere—*provided the sender generated* $(\mathsf{dpf}_0, \mathsf{dpf}_1)$ *honestly*, which the servers confirm via the audit protocol. In the audit protocol, the two servers engage in a *secret-shared non-interactive proof* (SNIP) to verify that $M_0 + M_1$ is *(at-most-)*1-hot. Notice that auditing requires $O(n\lambda^2)$ work—i.e., $O(n)$ evaluations of the DPF at $O(\lambda^2)$ work a piece—by each of server 0 and server 1.

1.4 De l'Épée au Sabre

We present Sabre, a family of SAM protocols with instances operating in both the sender-anonymous bulletin board model and the sender-anonymous mailbox model. Sabre protocols inherit much of their basic structure from Riposte and Express, but incorporate key innovations that improve not only concrete performance and scalability under "ideal" circumstances, but also resilience to resource exhaustion–style DoS attacks.

From a technical perspective, the primary difference is how Sabre implements auditing to identify malformed write requests: the *senders* construct compact SNIPs that "directly" attest to the well-formedness of the DPFs they submit. We construct the SNIPs in a novel model we call the *multi-verifier MPC-in-the-head* paradigm, which generalizes the (single-verifier) MPC-in-the-head paradigm of Ishai, Kushilevitz, Ostrovsky, and Sahai [15] to achieve good soundness at low cost in scenarios where two or more distinct verifiers can check two or more distinct subsets of simulated interaction transcripts. In the sender-anonymous mailbox model, Sabre also uses a new trick to decouple the cost of DPF evaluation from the bit length of the mailbox addresses. Together, these modifications yield speedups that exceed an order of magnitude relative to Express when all the write requests are "well-formed" and increase significantly—with the performance gap growing exponentially in the number of mailboxes—in the presence of resource-exhaustion DoS attacks.

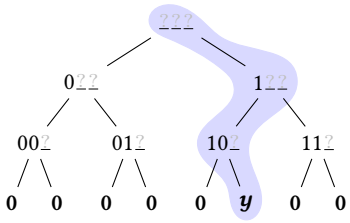


Figure 1: Binary-tree representation of the generalized point function with point $(x_5, y) \in D \times R$, $|D| = 2^3$.

1.5 Our Contributions

The main contribution of this paper is the design and evaluation of the Sabre family of SAM protocols. Sabre protocols are more performant than prior work by a (significant) constant factor when there is no attacker and they are *asymptotically faster* in the presence of DoS attacks. Toward the construction of Sabre, we make several additional contributions of independent interest, including

(1) an efficient secure $(2+1)$-party computation for the oblivious evaluation of $O(\lg n)$-sized DPFs;
(2) fast SNIPs from "multi-verifier" MPC-in-the-head;
(3) non-interactive zero-knowledge arguments of knowledge for the well-formedness of $O(\lg n)$-sized DPFs;
(4) parallel (bitsliced) implementations of fixed-key LowMC and $O(\lg n)$-sized DPF evaluation; and
(5) a pair of novel mailbox address–verification procedures with $O(n)$ and $O(1)$ complexity.

2 BUILDING BLOCKS

This section gives a brief overview of the various building blocks used to construct Sabre.

2.1 Distributed point functions

A *t-out-of-ℓ distributed point function*, or (t,ℓ)-DPF, is a compact (t,ℓ)-threshold shared representation of a so-called *(generalized) point function.* Sabre and the related SAM constructions we consider exclusively use $(2,2)$-DPFs (that is, the special case of $t = \ell = 2$); thus, for brevity, we generally omit the $(2,2)$- qualifier and refer merely to "DPFs".

2.1.1 (Non-distributed) point functions. Fix a domain $D \subseteq \{0,1\}^*$ and range R (a ring with unity), and let $\mathbf{0}$ and $\mathbf{1}$ denote the additive and multiplicative identities in R.

Definition 1. *The* point function at $x \in D$ over R *is the function* $P\colon D \to R$ *defined via*

$$P(i) := \begin{cases} \mathbf{1} & \text{if } i = x, \text{ and} \\ \mathbf{0} & \text{otherwise.} \end{cases}$$

That is, the point function at $x \in D$ over R is $\mathbf{0}$ everywhere except at x where it is $\mathbf{1}$. Alternatively, one can think of it as a "functional representation" of a vector from the standard basis of a $|D|$-dimensional module over R.

A "generalized" point function is just a point function scaled by some value from its range.

Definition 2. *If* $P\colon D \to R$ *is the point function at* $x \in D$ *over* R *and if* $y \in R$ *is a fixed scalar, then* $P_y\colon i \mapsto y \cdot P(i)$ *is the* generalized point function with point $(x, y) \in D \times R$.

Point functions as binary trees. Suppose $D \subseteq \{0,1\}^*$ is a well-ordered set with cardinality 2^h. One can naturally represent the generalized point function with point $(x, y) \in D \times R$ by a complete binary tree of height h having one leaf node labelled "y" and all other leaf nodes labelled "$\mathbf{0}$". To evaluate the point function on input $x_j \in D$—the "jth-smallest" element of D—begin at the root and parse the binary-string representation of j bit-by-bit from left to right, iteratively traversing either to the left child (if the next bit of j is 0) or to the right child (if the next bit of j is 1) of the current node until eventually arriving at a leaf. The label on this leaf defines the output of the point function on input x_j. Figure 1 depicts the binary-tree representation of the generalized point function with point $(x_5, y) \in D \times R$, where D has cardinality 2^3.

Before describing *distributed* point functions, we elaborate on some properties of such binary-tree representations that will prove useful in constructing the Sabre audit protocol. We first observe that such a tree comprises two distinct kinds of nodes, which we refer to as *type-0 nodes* and *type-1 nodes*. For brevity, we sometimes use the short hands *0-nodes* and *1-nodes* in reference to type-0 nodes and type-1 nodes, respectively.

Definition 3. *A node is of* type 0 *if it is (i) a leaf with label "$\mathbf{0}$", or (ii) a non-leaf and both of its children are of type 0; the node is of* type 1 *if it is (i) a leaf with a label other than "$\mathbf{0}$", or (ii) a non-leaf with one type-1 child and one type-0 child.*

The following observation is an immediate consequence of Definition 3.

Observation 1. *If a tree is rooted at a 0-node, all of its leaves are of type 0; if it is rooted at a 1-node, exactly one of its leaves is of type 1 and all others are of type 0.*

The proof of Observation 1 is by induction on the tree height h; it appears in Appendix A.

Observation 1 implies that any tree rooted at a 1-node contains exactly one path consisting entirely of 1-nodes from the root to some leaf node (and, moreover, that all 1-nodes in the tree reside along this path). We refer to such a path as a *type-1 path* through the tree (or a *1-path* for short). For example, the highlighted path in Figure 1 is the 1-path: every node in the highlighted path is of type 1 while every node not in the highlighted path is of type 0.

The above notions of 1-nodes and 1-paths give us some alternative ways to characterize the representation of generalized point functions as binary trees.

Corollary 1. *The following three characterizations are all equivalent: A complete binary tree of height h represents a generalized point function if and only if*

(1) exactly one leaf has type 1 and all others have type 0;
(2) its root has type 1; or
(3) it contains a 1-path of height h.

The first characterization in Corollary 1 is appealing because it most directly maps to the definition of a generalized point function (as stated in Definition 2); however, looking ahead, the third characterization will prove particularly convenient because it references just h nodes on a path from the root to a leaf, as opposed to all 2^h leaves. (The second refers just one node, yet the property of being a 1-node implicitly references a 1-path rooted at that node.)

2.1.2 2-out-of-2 distributed point functions. We now present a formal definition of $(2,2)$-DPFs based on those of Gilboa and Ishai [13] and Boyle, Gilboa, and Ishai [3, 4].

Definition 4. *A* $(2,2)$-distributed point function*, or* $(2,2)$-DPF*, is a pair of PPT algorithms* $(\mathsf{Gen}, \mathsf{Eval})$ *defining an infinite family of secret-shared representations of generalized point functions; that is, given (i) a security parameter* $\lambda \in \mathbb{N}$*, (ii) a domain* D *and range* R*, and (iii) a distinguished point* $(x, y) \in D \times R$*, we have*

1. **Correctness**: *If* $(\mathsf{dpf}_0, \mathsf{dpf}_1) \leftarrow \mathsf{Gen}(1^\lambda, D, R; x, y)$*, then, for all* $i \in D$*,*

$$\mathsf{Eval}(\mathsf{dpf}_0, i) + \mathsf{Eval}(\mathsf{dpf}_1, i) := \begin{cases} y & \text{if } i = x, \text{ and} \\ \mathbf{0} & \text{otherwise.} \end{cases}$$

2. **Simulatability**: *There exists a PPT simulator* $\mathcal{S}$ *such that, for any given domain* D*, range* R*, distinguished point* $(x, y) \in D \times R$*, and bit* $b \in \{0,1\}$*, the distribution ensembles*

$$\left\{\mathcal{S}(1^\lambda, D, R; b)\right\}_{\lambda \in \mathbb{N}}$$

and

$$\left\{\mathsf{dpf}_b \;\middle|\; (\mathsf{dpf}_0, \mathsf{dpf}_1) \leftarrow \mathsf{Gen}(1^\lambda, D, R; x, y)\right\}_{\lambda \in \mathbb{N}}$$

are computationally indistinguishable.

The dpf_b *output by* Gen *are* $(2,2)$-DPF keys*; the* x*-coordinate of the distinguished point is the* distinguished input.

2.1.3 The Boyle–Gilboa–Ishai DPF construction. The most compact DPF representation in the literature is due to Boyle, Gilboa, and Ishai [4]. Their construction uses a length-doubling PRG to define a pair of "garbled" binary trees that collectively represent some generalized point function. Let $G_{2\times}\colon \{0,1\}^* \to \{0,1\}^*$ be any length-doubling PRG and $G_{2^L}\colon \{0,1\}^* \to \mathbb{F}_{2^L}$ be a PRG with L-bit outputs. Boyle et al.'s construction follows from the simple observation that $G_{2\times}(r_0) = G_{2\times}(r_1)$ if and (essentially) only if $r_0 = r_1$; that is, length-doubling PRGs map additive shares of 0 to additive shares of $(0,0)$, while they map additive shares of $x \neq 0$ to additive shares of (y,z) such that *both* $y \neq 0$ *and* $z \neq 0$ with a probability overwhelming in $|x|$.

In the context of binary-tree representations of generalized point functions, this roughly equates to observing that $G_{2\times}$ preserves the property of being a 0-node. The challenge is now to extend this so that $G_{2\times}$ preserves node type for *all* nodes in the tree, be they the 0-nodes that comprise most of the tree or the 1-nodes along the path from the root to the distinguished point. To this end, the DPF keys include one λ-bit *correction word* (CW) per tree level, which is specially crafted to map *exactly one* child of the 1-node on that level to a 0-node. Finally, G_{2^L} "stretches" each node in the final (i.e., leaf) level into a pseudorandom L-bit value, and then a final L-bit CW maps the leaf-level 1-node to the desired output $y \in R$. We elaborate on the semantics of CWs when constructing our $(2+1)$-party computation for DPF auditing in Section 5; for now, we emphasize only that this approach yields *very* compact DPF keys, each comprising a length-$\lceil \lg|D| \rceil$ sequence of λ-bit values, plus one additional L-bit value to map the leaf-level 1-node to $y \in R$.

2.2 Secure multiparty computation

A *secure multiparty computation* (MPC) protocol is a cryptographic protocol through which two or more mutually distrusting parties jointly compute some function over their private inputs while disclosing nothing about their inputs beyond what is deducible from the shared output. In addition to the functionality being computed, MPC protocols are typically parameterized by (i) the number "ℓ" of parties to the computation, and (ii) the size "$t+1$" of coalition needed to violate the privacy of non-coalition members' inputs.

2.2.1 Secure $(2+1)$-party computation. One special case of MPC is so-called *secure* $(2{+}1)$*-party computation* (also known as *server-aided 2-party computation*), wherein two mutually distrusting parties enlist the help of a semi-honest third party (who provides no input and receives no output) to assist in the computation. Such protocols have received considerable attention due to their superior performance relative to "pure" 2-party protocols or ℓ-party protocols secure against stronger attackers. The Sabre audit protocols leverage $(2+1)$-party computation based on Du–Atallah multiplication [10], a brief description of which appears in Appendix B.

2.2.2 MPC-in-the-head. *MPC-in-the-head* [5, 12, 15, 16] is a conceptual framework for constructing ZKPoKs from MPC building blocks. ZKPoKs based on MPC-in-the-head leverage the dual observations that (i) MPC protocols are *simulatable*—even from the perspective of an "insider" (i.e., an attacker who controls a size-t subset of the parties to the computation)—and (ii) dishonest behaviour by a coalition in an MPC protocol execution necessarily yields incontrovertible evidence in the coalition members' joint view.

Consider an MPC protocol Π for the witness-checking procedure of some NP relation R. In light of the preceding two observations, one can transform Π into a ZKPoK for R as follows: Given a (public) instance I and (private) witness w such that $R(I,w) = 1$, the *prover* secret shares w among ℓ imaginary computation parties, and then it *simulates* a complete run of Π among these imaginary parties, up to and including the point where the functionality attests to the validity of w by outputting 1. The prover *commits* (say, via Merkle tree) to the simulated view of each imaginary party, presenting the resulting commitments to the verifier.

If the prover is attempting to cheat, then incontrovertible evidence must exist in at least one coalition's view; thus, the verifier attempts to uncover such evidence by challenging the prover to reveal the views of a random size-t coalition of parties. To amplify the probability of detecting cheating—a probability that varies based on the collusion threshold t and number of imaginary parties ℓ—the prover and verifier engage in multiple parallel instances of this procedure, each one operating on a "fresh" simulation.

2.2.3 The LowMC block cipher. LowMC [1] is a block cipher specially designed for secure computation settings like MPC protocols based on linear secret sharing, fully homomorphic encryption, or ZKPoKs. In such applications, linear operations are generally regarded as "free", while non-linear operations (e.g., the multiplication of two or more unknowns) are costly; thus, LowMC strives to balance multiplicative complexity and depth on one hand with concrete security on the other. In the context of $(2+1)$-party computation (and associated MPC-in-the-head), multiplicative complexity dictates communication cost (transcript size) while multiplicative depth dictates round complexity (dependency-chain length for the verifier). Jumping ahead, Sabre employs LowMC to construct the length-doubling PRGs in its DPFs. Several tuning knobs allow protocol designers to tailor LowMC's performance for an intended application; we empirically measure various knob settings for Sabre auditing in Section 6.1.

3 SYSTEM DESIGN

This section presents the system design of Sabre in both the sender-anonymous bulletin board model (Sabre-BB) and the sender-anonymous mailbox model (Sabre-M). In the latter model, we describe four distinct versions: One that closely mimics the design of Express; a second that adaptively modifies the mapping between DPFs and mailbox addresses to improve efficiency; and a third and fourth that push the optimization in the second version to its logical extreme.

The second through fourth versions necessitate new ways to ensure writers know valid mailbox addresses.

Each Sabre variant is instantiable in three ways (deriving from three ways to instantiate the audit protocol); namely, as (i) a 2-server version using "2-verifier MPC-in-the-head", (ii) a 3-server version using "3-verifier MPC-in-the-head", or (iii) a 3-server version directly using secure (2+1)-party computation. Except where otherwise specified, all comparisons of Sabre's performance to those of Riposte and Express refer to a 2-server instantiation—which provides the *strongest security guarantees* but the *poorest performance* of the three options. That said, we begin with high-level descriptions that abstract auditing into an inscrutable black box (thus relieving us of the need to describe all $3 \cdot (4+1) = 15$ possible combinations separately), before circling back to flesh out the details of auditing in Section 5. The reader should bear in mind that Sabre auditing requires just $O(\lambda \lg n)$ work to the $\Omega(\lambda n)$ work required by Riposte and Express auditing, and that Sabre auditing renders judgement *before* the servers ever "evaluate" their DPF keys (a linear-cost operation).

3.1 Sabre for sender-anonymous bulletin boards

Consider a (2- or 3-server) Sabre-BB instance with security parameter $\lambda \in \mathbb{N}$ (say, $\lambda = 128$) and a bulletin board comprising $n = 19.5m$ buckets each capable of holding a single L-bit message. (Thus, we have domain $D = [0..n-1]$ and range $R = \mathbb{F}_{2^L}$.) The design of Sabre-BB tightly parallels that of Riposte, save for the adoption of Boyle et al.'s more compact DPFs and the new audit protocol: A sender who wishes to post a message $M \in \mathbb{F}_{2^L}$ to the bulletin board

(1) samples a random bucket index $i \in_R D$;
(2) samples $(\mathsf{dpf}_0, \mathsf{dpf}_1) \leftarrow \mathsf{Gen}(1^\lambda, D, R;\ i, M)$; and then
(3) sends dpf_b to server b for $b = 0, 1$.

Upon receiving dpf_b, server b constructs the vector $M_b \in (\mathbb{F}_{2^L})^{1\times n}$ in which the jth component equals $\mathsf{Eval}(\mathsf{dpf}_b, j)$ for $j = 0, \ldots, n-1$; server b adds M_b to its bulletin board database to effectuate the write. We remark that M_b is efficiently computable (with a cost dominated by $2n$ length-doubling PRG evaluations and n evaluations of the leaf-stretching PRG) using the so-called *full-domain evaluation* procedure described by Boyle et al. [4, §3.2.3].

As per Definition 4, $M_0 + M_1 \in (\mathbb{F}_{2^L})^{1\times n}$ has message M in its ith column and zero elsewhere—*provided the sender generated* $(\mathsf{dpf}_0, \mathsf{dpf}_1)$ *honestly*, which the servers confirm via any of the three audit protocol instantiations.

3.2 Sabre for sender-anonymous mailboxes

We present four distinct variants of (2- or 3-server) Sabre-M, the first of which closely follows the design of Express with the other three implementing progressively more sophisticated optimizations to the mapping between DPFs and the registered mailbox addresses.

3.2.1 Sabre-M0: Express with improved auditing. Consider a Sabre-M0 instance with security parameter $\lambda \in \mathbb{N}$ (say, $\lambda = 128$). The design of Sabre-M0 tightly parallels that of Express, save for the new audit protocol. In a registration phase, the recipient contacts the servers to create a mailbox and receives a uniform random, λ-bit address in exchange. Suppose there are n registered mailboxes with addresses $addr_1, \ldots, addr_n$. To deposit a message $M \in \mathbb{F}_{2^L}$ into the mailbox addressed by $addr_i$, the sender samples $(\mathsf{dpf}_0, \mathsf{dpf}_1) \leftarrow \mathsf{Gen}(1^\lambda, \mathbb{F}_{2^\lambda}, \mathbb{F}_{2^L};\ addr_i, M)$ and then it sends dpf_b to server b for $b = 0, 1$.

Upon receiving dpf_b, server b constructs the vector $M_b \in (\mathbb{F}_{2^L})^{1\times n}$ in which the jth component equals $\mathsf{Eval}(\mathsf{dpf}_b, addr_j)$; server b adds M_b to its mailbox database to effectuate the write. We stress that (as in Express —owing to the sparsity of the set of mailbox addresses within $\{0,1\}^\lambda$) Boyle et al.'s *full-domain evaluation* procedure does not apply when computing M_b. Instead, each column requires $O(\lambda)$ length-doubling PRG evaluations (for a total of $O(\lambda n)$ evaluations), plus one evaluation of the leaf-stretching PRG.

As per Definition 4, $M_0 + M_1 \in (\mathbb{F}_{2^L})^{1\times n}$ has message M in its ith column and zero elsewhere—*provided the sender generated* $(\mathsf{dpf}_0, \mathsf{dpf}_1)$ *honestly*, which the servers confirm via any of the three audit protocol instantiations. Sabre-M0 very quickly rejects malformed DPFs outright, whereas (like in Express) otherwise well-formed DPFs whose distinguished points do not correspond to registered mailbox addresses will pass auditing, ultimately resulting in a (comparatively expensive) no-operation "update" to the mailbox database. In particular, such well-formed, yet bogus writes incur the same *decidedly non-trivial* cost as "true" writes.

3.2.2 Sabre-M1: Sabre-M0 with shorter DPFs. Sabre-M1 provides a modest improvement over Sabre-M0 by dynamically "truncating" mailbox addresses to a prefix just long enough to avoid collisions with high probability, thereby reducing both (i) the size of (and cost of auditing) the keys dpf_0 and dpf_1 and (ii) the cost of computing M_0 and M_1 from them.

Upon registering a mailbox, the recipient still receives a uniform random, λ-bit mailbox address that senders must know (in its entirety) to deposit messages into the mailbox. The difference is that now only a prefix of this address (whose length grows with the number of registered mailboxes) is used when sampling and evaluating DPFs. The intuition here is that—for purposes of correctness—it suffices merely to guarantee that no two mailboxes map to the same prefix; thus, one can compute the required prefix length for a given number of registered mailboxes using a standard birthday bound calculation. (In our experiments in Section 6.3.1, we chose the length to ensure that the probability of one or more prefix collisions is strictly less than 0.001.)

This optimization improves efficiency relative to Sabre-M0. However, it comes at the cost of increasing the probability of an attacker "guessing" a distinguished point corresponding to a valid mailbox—that is, random guesses potentially clobber a registered mailbox with a probability that is now *polynomial*

in n. To mitigate, Sabre-M1 requires the sender to transmit *an additive share of the full mailbox address* alongside dpf_b, after which the servers use a simple PIR-based protocol to verify that the address shares reconstruct to the full address of whatever mailbox the DPF keys reference. We describe the address-verification protocol in Section 4.

The *full-domain evaluation* procedure still does not apply when computing M_b in Sabre-M1; however, now the cost of computing each column reduces from $\mathsf{O}(\lambda)$ to just $\mathsf{O}(\lg n)$ length-doubling PRG evaluations (reducing the total cost of computing M_b from $\mathsf{O}(n\lambda)$ to just $\mathsf{O}(n \lg n)$ evaluations), plus n evaluations of the leaf-stretching PRG. Furthermore, the servers can now detect and reject DPFs from senders who do not know a registered mailbox address *without incurring a costly no-operation to "update" the database.*

3.2.3 Sabre-M2: Sabre-M1 with decoupled address verification.

Sabre-M2 improves on Sabre-M1 by eliminating the association between DPF leaves and mailbox addresses: Instead of sampling DPFs whose distinguished points correspond to mailbox addresses, senders sample DPFs whose distinguished points correspond to the *chronological order in which those mailboxes were registered*. In other words, to deposit a message $M \in \mathbb{F}_{2^L}$ into the mailbox with address $addr_i$, a sender samples keys for a DPF with point $(i, M) \in [0\,..\,n-1] \times \mathbb{F}_{2^L}$ rather than $(addr_i, M) \in \mathbb{F}_{2^\lambda} \times \mathbb{F}_{2^L}$. With this optimization, (i) the domain of DPFs has size equal to the number of registered mailboxes, and (ii) the *full-domain evaluation* procedure applies when computing the M_b. This reduces the total cost of computing M_b to just $2n$ length-doubling PRG evaluations plus n evaluations of the leaf-stretching PRG. To ensure that senders know the correct mailbox address, Sabre-M2 inherits the PIR-based address-verification technique of Sabre-M1.

3.2.4 Sabre-M: Sabre-M2 with $\mathsf{O}(1)$-time address verification.

Sabre-M improves on Sabre-M2 by replacing the linear-complexity PIR-based address-verification procedure with a constant-complexity *PRF-based* one described in Section 4. With this modification in place, the entire auditing procedure—that is, both checking the well-formedness of the DPF *and* checking the validity of the mailbox address—has complexity logarithmic in n, allowing the servers to rapidly reject bogus write requests from a would-be resource-exhaustion DoS attacker. The benefits of rapid auditing bare out in our experimental evaluation in Section 6.

Summary of Sabre-M variants. We conclude this section by recapping the key difference among Sabre-M instances.

(1) Sabre-M0 follows Express in checking DPF well-formedness while ignoring address validity. This approach performs well when most DPFs are either malformed or write to registered addresses, but it is inefficient for DPFs targeting unregistered mailbox addresses.
(2) Sabre-M1 reduces the size of the DPFs, improving the rate at which both well-formed and malformed write requests can be processed. It introduces a lightweight mailbox address check that efficiently rejects incoming requests not targeting registered mailboxes.
(3) Sabre-M2 further reduces the size of DPFs, yielding additional speedups for both well-formed and malformed requests. It uses Sabre-M1's mailbox address check.
(4) Sabre-M replaces the linear-cost mailbox address check in Sabre-M2 with a constant-cost PRF-based check. This approach performs well in all cases.

4 VERIFYING MAILBOX ADDRESSES

This section describes how Sabre servers in the sender-anonymous mailbox model verify that incoming requests target valid mailbox addresses (excepting Sabre-M0, which follows Express in forgoing explicit address verification). Due to space constraints, we describe here only the high-level design and relegate full details to Appendix H.

The idea is quite simple: Along with dpf_b, the sender transmits a $(2, 2)$-additive share $addr_b \in \mathbb{F}_{2^\lambda}$ of the mailbox address $addr$ to server b for $b = 0, 1$. The next step differs between the two versions of the protocol:

(1) with PIR verification, the servers use their respective DPF keys to *obliviously fetch* the mailbox address associated with the DPF's distinguished point; whereas,
(2) with PRF verification, the servers re-purpose audit-protocol inputs to *obliviously compute* the mailbox address associated with the DPF's distinguished point.

In both versions, the servers now hold two independent $(2, 2)$-additive sharings: one, $addr_b$, of the *sender-claimed* address and another, $addr'_b$, of the *server-computed* address. Server b computes $A_b := addr_b + addr'_b$, and they check if

$$\begin{aligned} A_0 + A_1 &= (addr_0 + addr'_0) + (addr_1 + addr'_1) \\ &= (addr_0 + addr_1) + (addr'_0 + addr'_1) \\ &= addr + addr' \\ &= \mathbf{0}, \end{aligned}$$

where the last equality holds only if $addr = addr'$. In particular, the check succeeds if and only if $A_0 = A_1$, which the servers can confirm alongside the final step of the audit protocol.

5 THE SABRE AUDIT PROTOCOL

The *audit protocol* verifies the well-formedness of DPF keys submitted by writers in both sender-anonymous bulletin board and sender-anonymous mailbox variants of Sabre. It is instantiable in three distinct ways: as (i) a secure (2+1)-party computation, (ii) a 3-verifier MPC-in-the-head SNIP, or (iii) a 2-verifier MPC-in-the-head SNIP. The latter two instantiations build on the first.

5.1 Secure $(2+1)$-party auditing

Following Corollary 1, the secure $(2+1)$-party audit protocol demonstrates the existence of a 1-path from the root of the

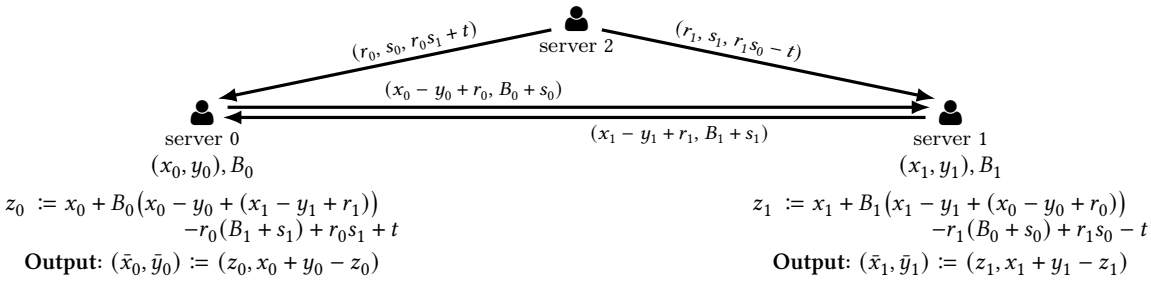


Figure 2: $(2+1)$**-party realization of the conditional swap functionality** $([\bar{x}], [\bar{y}]) \leftarrow \mathcal{F}_{\mathsf{SWAP}}\big(([x], [y]), [B]\big)$.

binary-tree representation of a DPF to the leaf corresponding to its distinguished point. It does so via an alternating sequence of two simple functionalities: (i) oblivious length–doubling PRG evaluation and (ii) conditional swapping on the components of an ordered pair.

5.1.1 Oblivious PRG. The oblivious PRG evaluation functionality $([x], [y]) \leftarrow \mathcal{F}_{\mathsf{PRG}}([z])$ uses the Matyas–Meyer–Oseas (MMO) one-way compression function [21]: Given a block cipher Enc with λ-bit blocks, define $G_{2\times}\colon \{0,1\}^\lambda \to \{0,1\}^\lambda \times \{0,1\}^\lambda$ via

$$G_{2\times}(z) := \big(\mathsf{Enc}_K(z) \oplus z,\, \mathsf{Enc}_K(z \oplus 1) \oplus (z \oplus 1)\big)\,, \qquad (1)$$

where K is a fixed (publicly known) key. Sabre uses fixed-key LowMC as the block cipher in Equation (1). We remark that using a fixed, publicly known key in LowMC eliminates all non-linear operations outside of the S-boxes, and it provides new opportunities for preprocessing-based optimizations involving round-key matrices. The S-boxes operate on ordered triples of bits, and each S-box in a round can be evaluated (in parallel) using just three 1-bit multiplications a piece, leveraging their representation in algebraic normal form [1, Appendix C]:

$$(a, b, c) \mapsto (a \oplus bc,\, a \oplus b \oplus ac,\, a \oplus b \oplus c \oplus ab)\,.$$

5.1.2 Conditional swapping. The conditional swap functionality $([\bar{x}], [\bar{y}]) \leftarrow \mathcal{F}_{\mathsf{SWAP}}\big(([x], [y]), [B]\big)$ uses the standard trick for conditional branching in MPC. Players 0 and 1 respectively hold $(2,2)$-additive shares of (i) an ordered pair (x, y) and (ii) a selection bit B on which to condition the swap. They compute shares $[z]_0$ and $[z]_1$ of $z = x + B(y - x)$, and then output $([\bar{x}], [\bar{y}]) = ([z], [x] + [y] - [z])$. An easy calculation verifies that $(\bar{x}, \bar{y}) = (x, y)$ if $B = 0$ and $(\bar{x}, \bar{y}) = (y, x)$ if $B = 1$. Figure 2 illustrates the conditional swap protocol.

5.1.3 Non-private DPF traversal. Recall from Section 2.1.3 that Boyle–Gilboa–Ishai DPF keys have the form

$$\mathsf{dpf}_b := (\mathsf{seed}_b, \mathsf{cw}^{(1)}, \ldots, \mathsf{cw}^{(h)}) \in (\mathbb{F}_{2^\lambda})^h \times \mathbb{F}_{2^L}\,,$$

where h is the tree height. The correction words $\mathsf{cw}^{(1)}, \ldots, \mathsf{cw}^{(h)}$ are common to both keys dpf_0 and dpf_1 in a key pair.

Parse seed_b as $(s_b^{(0)}, \cdot, \mathsf{flag}_b^{(0)}) \in \mathbb{F}_{2^{\lambda-2}} \times \mathbb{F}_2 \times \mathbb{F}_2$; we regard the latter tuple as a "label" for the tree root. To traverse a node with label $(s_b^{(j-1)}, \cdot, \mathsf{flag}_b^{(j-1)}) \in \mathbb{F}_{2^{\lambda-2}} \times \mathbb{F}_2 \times \mathbb{F}_2$ in level $j-1$ of the tree to its Bth child in level $j \in [1..h]$:

(1) compute $(\mathsf{child}_{b,0}, \mathsf{child}_{b,1}) \leftarrow G_{2\times}(s_b^{(j-1)} \| 00)$;
(2) parse $\mathsf{child}_{b,B}$ as $(\bar{s}_b, \cdot, T_b) \in \mathbb{F}_{2^{\lambda-2}} \times \mathbb{F}_2 \times \mathbb{F}_2$ and $\mathsf{cw}^{(j)}$ as $(\overline{\mathsf{cw}}^{(j)}, t_1^{(j)}, t_0^{(j)}) \in \mathbb{F}_{2^{\lambda-2}} \times \mathbb{F}_2 \times \mathbb{F}_2$;
(3) compute $s_b^{(j)} \leftarrow \bar{s}_b \oplus \big(\overline{\mathsf{cw}}^{(j)} \cdot (1 \oplus \mathsf{flag}_b^{(j-1)})\big)$ and $\mathsf{flag}_b^{(j)} \leftarrow T_b \oplus (t_B^{(j)} \wedge \mathsf{flag}_b^{(j-1)})$; and then
(4) output $(s_b^{(j)}, \cdot, \mathsf{flag}_b^{(j)}) \in \mathbb{F}_{2^\lambda} \times \mathbb{F}_2 \times \mathbb{F}_2$ as the child's label.

Our $(2+1)$-party auditing protocol follows from the second characterization of a point function in Corollary 1. Suppose the DPF has distinguished input i having 0-padded binary representation $i = (B^{(1)}\ B^{(2)}\ \cdots\ B^{(h)})_2$. Then, given their respective DPF keys together with bitwise $(2,2)$-additive shares of i, the players (i) traverse to both children of the root using the above procedure, (ii) reconstruct the $(1 - B^{(1)})$th child to ensure it is a 0-node, and then (iii) recurse on the height-$(h-1)$ tree rooted at the $(B^{(1)})$th child.

If any $(1 - B^{(j)})$th child along the path from the root to a leaf is not of type 0, then i does not define a 1-path and the players reject the key pair.

5.1.4 MPC-based auditing. The $(2+1)$-party audit protocol implements the strategy just described via alternating applications of the oblivious PRG functionality (§5.1.1) and conditional swap functionality (§5.1.2), woven together with Du–Atallah multiplication (§2.2.1).

For each $b = 0, 1$, player b receives as input (i) its DPF key $\mathsf{dpf}_b := (\mathsf{seed}_b, \mathsf{cw}^{(1)}, \ldots, \mathsf{cw}^{(h)})$ and (ii) bitwise $(2,2)$-additive shares $[\![i]\!] := ([B^{(1)}], \ldots, [B^{(h)}])$ of the (purported) distinguished input i. Player 2 assists with the computation but receives no input and produces no output.

To begin, player b uses its inputs and $G_{2\times}$ to *non-obliviously* traverse to both children of the root; i.e., it computes

$$(x_b^{(1)}, y_b^{(1)}) := \big((s_{b,0}^{(1)}, \cdot, \mathsf{flag}_{b,0}^{(1)}), (s_{b,1}^{(1)}, \cdot, \mathsf{flag}_{b,1}^{(1)})\big)$$

via Steps (1)–(4) from Section 5.1.3, and then it secret shares both halves of the output with player $(1-b)$.

Upon receiving shares from player $(1-b)$, player b now holds both $([x_0^{(1)}]_b, [y_0^{(1)}]_b)$ and $([x_1^{(1)}]_b, [y_1^{(1)}]_b)$. Enlisting the help of player 2, for each $j = 1, \ldots, h-1$, the pair then:

(1) invoke $\mathcal{F}_{\mathsf{SWAP}}$ twice to produce
  - $([\bar{x}_0^{(j)}], [\bar{y}_0^{(j)}]) \leftarrow \mathcal{F}_{\mathsf{SWAP}}\big(([x_0^{(j)}], [y_0^{(j)}]), [B^{(j)}]\big)$
  - $([\bar{x}_1^{(j)}], [\bar{y}_1^{(j)}]) \leftarrow \mathcal{F}_{\mathsf{SWAP}}\big(([x_1^{(j)}], [y_1^{(j)}]), [B^{(j)}]\big)$;
(2) reconstruct $x^{(j)} := \bar{x}_0^{(j)} \oplus \bar{x}_1^{(j)}$ and abort if $x^{(j)} \neq 0^\lambda$; else parse

- $[\bar{y}_0^{(j)}]$ as $([\bar{s}_0^{(j)}], \cdot, [\mathsf{flag}_0^{(j)}])$
- $[\bar{y}_1^{(j)}]$ as $([\bar{s}_1^{(j)}], \cdot, [\mathsf{flag}_1^{(j)}])$;

(3) invoke $\mathcal{F}_{\mathsf{PRG}}$ twice to produce
- $([\mathsf{child}_{0,0}], [\mathsf{child}_{0,1}]) \leftarrow \mathcal{F}_{\mathsf{PRG}}([\bar{s}_0^{(j)}] \| 00)$
- $([\mathsf{child}_{1,0}], [\mathsf{child}_{1,1}]) \leftarrow \mathcal{F}_{\mathsf{PRG}}([\bar{s}_1^{(j)}] \| 00)$;

(4) parse each of
- $([\mathsf{child}_{0,0}], [\mathsf{child}_{0,1}])$ as $\big(([\bar{s}_{0,0}], \cdot, [T_{0,0}]), ([\bar{s}_{0,1}], \cdot, [T_{0,1}])\big)$
- $([\mathsf{child}_{1,0}], [\mathsf{child}_{1,1}])$ as $\big(([\bar{s}_{1,0}], \cdot, [T_{1,0}]), ([\bar{s}_{1,1}], \cdot, [T_{1,1}])\big)$
- $\mathsf{cw}^{(j+1)}$ as $(\overline{\mathsf{cw}}^{(j+1)}, t_1^{(j+1)}, t_0^{(j+1)})$;

(5) for each $b = 0, 1$ and $B = 0, 1$, compute
- $[s_{b,B}^{(j+1)}] \leftarrow [\bar{s}_{b,B}] \oplus \big(\overline{\mathsf{cw}}^{(j+1)} \cdot (b \oplus [\mathsf{flag}_b^{(j)}])\big)$,
- $[\mathsf{flag}_{b,B}^{(j+1)}] \leftarrow [T_{b,B}] \oplus (t_{B^{(j+1)}} \wedge [\mathsf{flag}_b^{(j)}])$; and,

(6) finally, set
- $([x_0^{(j+1)}], [y_0^{(j+1)}]) \leftarrow \big(([s_{0,0}^{(j+1)}], \cdot, [\mathsf{flag}_{0,0}^{(j+1)}]), ([s_{0,1}^{(j+1)}], \cdot, [\mathsf{flag}_{0,1}^{(j+1)}])\big)$
- $([x_1^{(j+1)}], [y_1^{(j+1)}]) \leftarrow \big(([s_{1,0}^{(j+1)}], \cdot, [\mathsf{flag}_{1,0}^{(j+1)}]), ([s_{1,1}^{(j+1)}], \cdot, [\mathsf{flag}_{1,1}^{(j+1)}])\big)$.

5.2 SNIP-based auditing

The preceding section described a $(2+1)$-party computation using which servers 0 and 1 can (with assistance from a semi-honest server 2) efficiently check the well-formedness of their respective Boyle–Gilboa–Ishai DPF keys. We now explain the conversion of this $(2+1)$-party computation into a zero-knowledge argument based on MPC-in-the-head with multiple verifiers.

The high-level idea is for the prover, who holds both dpf_0 and dpf_1 along with the distinguished input i, to *simulate* $(2+1)$-party auditing and then commit to the (unidirectional) communication channels over which the servers exchange messages in the simulation. From here, the verifiers challenge the prover to open subsets of these channel commitments for inspection.

5.2.1 MPC-in-the-head with multiple verifiers. A critical difference between our arguments and other zero-knowledge arguments constructed in the MPC-in-the-head paradigm stems from (i) our use of a $(2+1)$-party computation secure against a *single passive corruption*, combined with the fact that (ii) the private inputs to players 0 and 1 in the underlying MPC include the private inputs to servers 0 and 1 in Sabre (i.e., dpf_0 and dpf_1, respectively). Consequently, server b can scrutinize the view of *either* player b *or* player 2 from a given simulation, but *never both*—and it can *never* scrutinize the view of player $1-b$. This fact all but rules out sound "single-verifier" arguments, as a cheating prover could always, e.g., confine inconsistencies to transcripts that the verifier is not allowed to scrutinize.

We circumvent this issue by leveraging two or more verifiers that each examine a different subset of transcripts. The argument is accepted if and only if *all* verifiers (i) receive identical channel commitments from the prover, and (ii) find no inconsistencies in the views they scrutinize. Definitionally, such "multi-verifier" MPC-in-the-head arguments are equivalent to *secret-shared non-interative proofs* (*SNIPs*) with a (modestly) generalized zero-knowledge property: whereas Corrigan-Gibbs and Boneh's definition [8, §4.1] insists that the view of *any proper subset* of verifiers be simulatable, "multi-verifier MPC-in-the-head" SNIPs inherit their simulatability requirements from the access structures governing privacy in the underlying MPC. In the case of Sabre's $(2+1)$-party auditing, the requirement is that the view of *any given verifier* be simulatable. When there are just two verifiers, this requirement coincides with the one originally proposed by Corrigan-Gibbs and Boneh. We deal with this model in more detail in Appendix C.

5.2.2 Constructing the Proof. We can now present the proof construction do the SNIP-based auditing using 2-servers. In order to achieve a low soundness error, SNIP-based auditing employs the cut-and-choose technique. The prover runs K independent and parallel simulations of the MPC protocol describe in Section 5.1.4 in their head. Suppose that, (i) $\mathbf{m}_i[x, y]$, the ordered set of messages sent from server x to server y in the ith simulation, (ii) $\mathcal{H}(m)$ is the hash of message m, (iii) $\mathcal{H}(m_0, \cdots, m_k)$ is the hash of $m_0 \| \ldots \| m_k$, (iv) $\mathcal{H}_i[x, y]$ is the hash of ordered messages from server x to server y in the ith simulation, (v) s_i is the PRG seed used by server 2 in the ith simulation to generate the Du-Attalah multiplication triples to facilitate the MPC, and (vi) $\widehat{\mathcal{H}}_i$ is the short hand for $\mathcal{H}\Big(\mathcal{H}_i[0,1], \mathcal{H}_i[1,0], \mathcal{H}_i[2,0], \mathcal{H}_i[2,1]\Big)$.

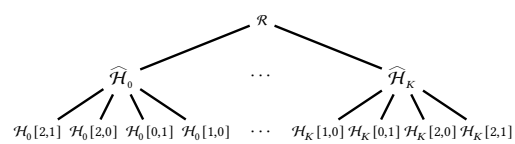


Figure 3: Merkle-tree Construction

To construct the proof, prover constructs a merkle-tree of depth 2 (see Figure 3). $\mathcal{H}_0[0,1]$, $\mathcal{H}_0[1,0]$, $\mathcal{H}_0[2,0]$, $\mathcal{H}_0[2,1]$, ..., $\mathcal{H}_K[0,1]$, $\mathcal{H}_K[1,0]$, $\mathcal{H}_K[2,0]$, $\mathcal{H}_K[2,1]$ are leaves of this tree. The penultimate layer of the merkle-tree is $\widehat{\mathcal{H}}_0, \ldots, \widehat{\mathcal{H}}_K$. Finally, $\mathcal{H}\Big(\widehat{\mathcal{H}}_0, \ldots, \widehat{\mathcal{H}}_K\Big)$ is the root of the merkle-tree. We denote the root as $\mathcal{R}$. Also, let $\mathcal{R}_i$ be the ith bit of $\mathcal{R}$. The proof for verifier server b consists of: (i) the root of the Merkle-tree, $\mathcal{R}$, (ii) For all i, such that $\mathcal{R}_i = 1$: $\mathbf{m}_i[1-b, b]$, $\mathbf{m}_i[2, b]$, $\mathcal{H}_i[2, 1-b]$, $\mathcal{H}_i[2, b]$, and (iii) For all i, such that $\mathcal{R}_i = 0$: $\mathcal{H}_i[1-b, b]$, $\mathcal{H}_i[1-b, b]$, s_i.

5.2.3 2-Server Verifier. The verification process for the two verifiers is symmetrical. At a high level, the two verifiers server 0 and server 1 try to reconstruct the merkle-tree root. If both of them succeed in doing so, they accept the proof. The verification process for verifier server b can be described as follows: (i) For i, such that $\mathcal{R}_i = 0$, the server b uses the s_i to construct $\mathbf{m}_i[2,0]'$ and $\mathbf{m}_i[2,1]'$, and (ii) For i, such that $\mathcal{R}_i = 1$, the Verifier server b uses $\mathbf{m}_i[1-b, b]$ and $\mathbf{m}_i[2, b]$ to

construct $\mathbf{m}_i[b, 1-b]'$. The verifiers can now construct the merkle-tree and thus verify that the root is indeed correct. The two verifiers in combination thus verify server 2 and server b's ($b = \{0, 1\}$) part of the proof. For soundness and completeness proofs, see Appendix E. Our implementations use several optimization techniques which we describe in Appendix F. In Appendix D, we show how a third verifier would give 128 time speed up in both the server and client costs.

6 IMPLEMENTATION AND EVALUATION

To assess the practicality of Sabre, we wrote a proof-of-concept reference implementation in C++. Our implementation uses Boost.Asio v1.18.1 for asynchronous communication, OpenSSL 1.1.1i for hashing and TLS support, and dpf++ [14] for $(2, 2)$-DPFs; we wrote all other non-STL functionality by hand.[1].

Experimental setup. We ran a series of experiments on Amazon EC2, with the servers running in geographically distant locations to mimic realistic Internet latency. The servers are all m5.4xlarge instances equipped with 64 GiB of RAM and 16 vCPUs, running Ubuntu 18.04. For each of the experiments reported in this section, we configured LowMC to use 128-bit blocks (and 128-bit keys) with $\mathsf{r} = 19$ rounds consisting of $\mathsf{s} = 32$ S-boxes a piece. Appendix J presents experimental results to justify these parameter choices. (As a spoiler, this setting turns out to be pessimal with respect to $(2+1)$-party auditing and SNIPs, but optimal for the full-domain evaluation, which dominates the execution time.) We ran all experiments for 100 trials and report in our plots the sample mean over those 100 runs.

6.1 Communication Cost

We first present an (analytically determined) accounting of communication costs for all three variants of Sabre auditing. The costs are summarized in Table 1, with the derivations of these numbers included as Appendix G. (Note that these costs account for *auditing only*; in particular, they do not include the DPF.) Observe that the client-to-server communication cost in both 2- and 3-verifier SNIP-based auditing scales with the product $\mathsf{r}\,\mathsf{s}$ of LowMC parameters. (To a first approximation, $\mathsf{r} \approx \lg \mathsf{s}$ so that proof-size scaling is roughly linear in r.) Also notice that 3-verifier SNIPs reduce communication about 64-fold relative to 2-verifier SNIPs; however, this reduction comes at the cost of an additional server and corresponding non-collusion assumption.

6.2 Auditing

Our first set of experiments measure the cost of Sabre auditing and compare it against that of Riposte and Express auditing. Figure 4 shows the relative cost of $(2+1)$-party auditing, 3-verifier SNIP auditing, and 2-verifier SNIP auditing.

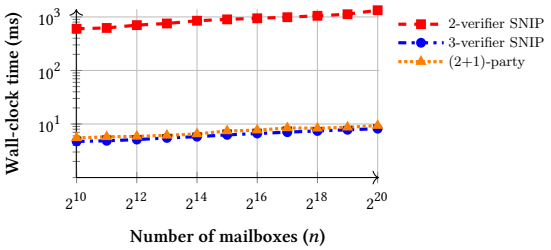


Figure 4: Time to audit 128 requests in Sabre

Owing to its reliance on cut-and-choose with 128 parallel instances, the 2-verifier SNIP auditing is by far the slowest of the pack, taking two orders of magnitude longer than both $(2+1)$-party and 3-verifier SNIP auditing. Nevertheless, the amortized cost per audit still barely exceeds 10 ms for Sabre instances even with $n = 2^{20}$ registered mailboxes. Although 2-verifier SNIP auditing is dramatically less performant than the alternatives, it offers superior security guarantees; thus, we restrict our remaining experimental results to this variant. It is worth noting, however, that either 3-server variant would provide (significantly) better performance wherever an additional party (and the resulting stronger trust assumption) is palatable.

Figures 5a and 5b compare the audit costs of (2-verifier) Sabre, Riposte, and Express, while Figure 5a, 5c zeros in on the cost of verifying mailbox addresses using both the PIR-based and PRF-based checks.[2] The first two plots clearly illustrate the benefits of logarithmic auditing relative to the linear-cost auditing in Riposte and Express. While the latter perform better for very small DPFs, the superior scaling of Sabre's audit protocol becomes significant as DPFs begin to exceed around a hundred thousand mailboxes.

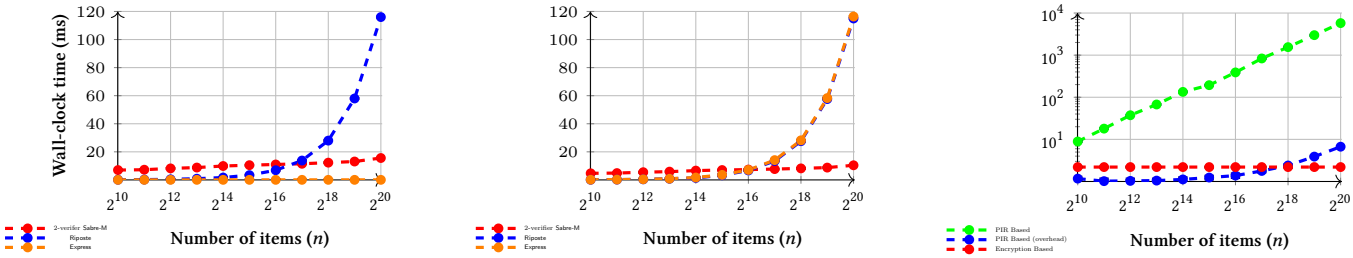
Note that the PIR-based mailbox check has two components: (i) full-domain evaluation of the DPF, and (ii) using the result to perform a PIR query. Since the former step is required (for valid requests) regardless, we include a separate plot for the latter, which corresponds to the *overhead* imposed by PIR-based mailbox checking. We see from Figure 5c that as expected, the PRF-based mailbox address check is independent of the number mailboxes. The figure also shows that when the number of mailboxes is small, the overhead cost of doing the PIR based DPF check is lower than that of the PRF-based mailbox address check. However, the linear growth of the PIR-based check's overhead ensures that, for larger DPFs, the overhead cost to do the PIR-based check overtakes the cost of doing the encryption-based mailbox-check. When we account for the full-domain evaluation of the DPF—which needs to happen before we can reject a malformed request in the PIR-based check, but not in the

[1] Our source code is available for anonymous review via https://github.com/secure-systems/Sabre; it will be made publicly available under the GNU General Public License (version 3) upon acceptance.

[2] The x-axis in all three plots corresponds to the number of size (i.e., number of leaves) of the DPF. (Thus, to interpret these results in the mailbox model, divide n by 19.5.)

Table 1: Communication cost of Sabre auditing (in bytes). In the table, n denotes the number of buckets/mailboxes, while r and s respectively denote the number of rounds and S-boxes in LowMC with 128-bit blocks. The 2-verifier SNIP employs cut-and-choose with soundness error $\approx 2^{-128}$.

Audit type	client→server b	client→server 2	server b ↔server $(1-b)$	server b →server 2	server 2→server b
(2+1)-party	—	—	$16+(16\frac{1}{8}+\frac{3}{2}\,\mathsf{s}\,\mathsf{r})(h-1)$	$16+(16\frac{1}{8}+\frac{3}{2}\,\mathsf{s}\,\mathsf{r})(h-1)$	$16+(32\frac{1}{8}+\frac{3}{2}\,\mathsf{s}\,\mathsf{r})(h-1)$
3-verifier SNIP	$112+\lceil(64.25+3\,\mathsf{s}\,\mathsf{r})(h-1)\rceil$	64	16	16	16
2-verifier SNIP	$37888+(4112+192\,\mathsf{s}\,\mathsf{r})(h-1)$	—	16	—	—



(a) Client-side auditing time (b) Server-side auditing time (c) Mailbox verification time

Figure 5: Comparing Sabre's audit protocols with Riposte and Express's audit protocols. Note that y-axis of Figure 5c log-scaled.

PRF-based check—the PRF-based check is always faster. While processing "good" writes, the time to do the audit is not the bottleneck and does not effect the overall running time. A faster audit protocol is useful in rejecting "bad" write requests.

6.3 Comparisons with the state-of-the-art

This section makes head-to-head comparisons of Sabre against Express and Riposte; the state-of-the-art in the "mailbox" and the "bulletin board" models respectively.

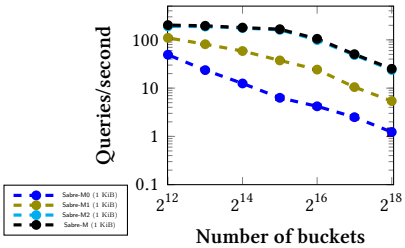
6.3.1 Head-to-Head with Express.

"Good" Writes. Figure 6a compares the different versions of Sabre when all the queries are "good" (i.e. they pass both the mailbox check and the DPF validity check). The progressive improvement of the different versions of Sabre is due the fact, that the height of the DPF tree progressively decreases. The main difference between Sabre-M2 and Sabre-M (who have the same DPF height) is that the PIR-based (linear time) "mailbox check" in Sabre-M2 is replaced by an encryption-based (constant time) "mailbox check". While processing good writes the DPF evaluation is the bottle-neck and hence we see that Sabre-M and Sabre-M2 have a similar performance. Since Sabre-M is the most performant version of Sabre, Figure 6b compares Sabre-M against Express for different message sizes when all the queries are "good". We note Sabre-M can processes more queries per second with messages of size 32 KiB, than Express can with messages of 1 KiB. The primary reason for Sabre-M being more performant than Express while processing "good" writes is that, Sabre-M's novel disruption resistance allows for DPF trees to have a much smaller height compared to Express.

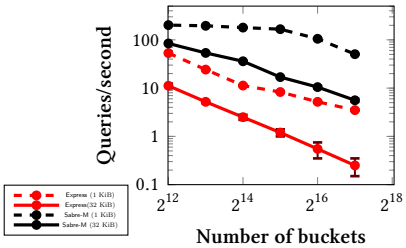
"Bad" Writes. Recall that a write request in the "mailbox" model is considered malformed if one of the following is true. (i) The DPF is malformed, and (ii) The DPF is indexed at a mailbox id which is not registered. Figure 7 compares the different versions of Sabre against Express by varying the percentage of malformed queries by keeping the number of mailboxes constant. We do this comparison for different ratios of bad-dpfs and bad-mailboxes. Notice that when "bad" DPFs form a large percentage of the "bad" writes, Sabre-M0, Sabre-M1, and Sabre-M2 perform much better than when "bad" mailboxes account for a larger percentage. This is due to the fact that, for Sabre-M0, Sabre-M1, and Sabre-M2 in order to reject a query with "bad" DPF, the DPF need not be expanded. However, when there is a query with a "bad" mailbox, DPF has to be expanded to do the PIR-based mailbox check, before it can be rejected. Note that Sabre-M (the final version of Sabre) is oblivious to the type of malformed query. Sabre-M, unlike its stepping stone versions, uses a LowMC encryption to generate the mailbox ids. Therefore, Sabre-M can do "mailbox check" in constant time and "dpf check" in logarithmic time. In summary, the performance improvement of Sabre-M for processing "bad" queries comes due to the fact that, (i) Sabre-M unlike Express can reject "bad" queries with expanding them. In other words, Sabre needs to do only $O(\lg n)$ work to reject a "bad" query as opposed to $O(n)$ in Express, and (ii) Sabre-M uses much smaller DPFs compared to Express.

6.3.2 Head-to-Head with Riposte.

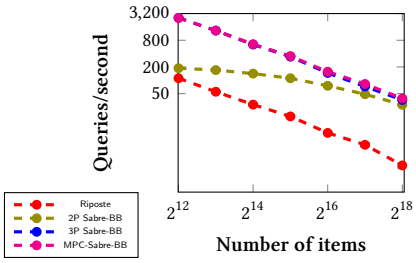
In Figure 6c, we compare different versions of Sabre-BB against Riposte. When we compare the $(2+1)$-secure computation based Sabre-BB against Riposte, we notice that Sabre-BB is nearly 25 times faster



(a) Comparing the different versions of Sabre with each other when there are 100% "good" writes



(b) Comparing Express against the different versions of 2-Party-Sabre-M, with 100% "good" writes; the y-axis is log-scaled



(c) Comparing Riposte against different versions Sabre-BB with message size of 256 bytes, with 100% "good" writes; y-axis is log scaled

Figure 6: Comparing Sabre-BB and Sabre-M against Riposte and Express respectively, when we have 100% "good" writes; the y-axis is log-scaled

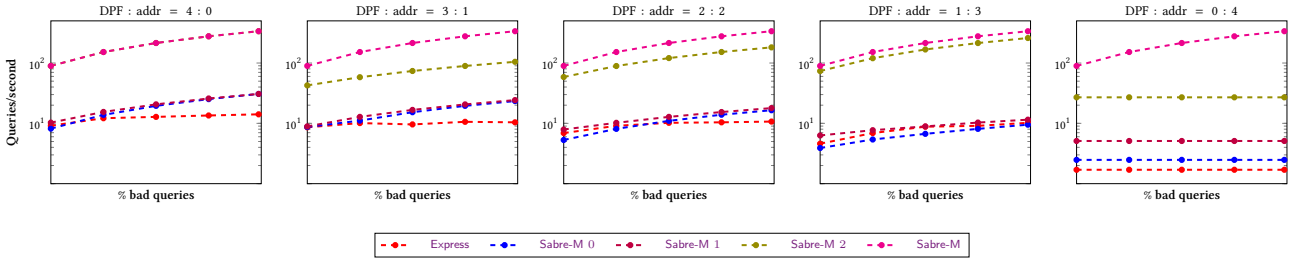


Figure 7: Varying the percentage of malformed queries for DPFs of size 2^{16} and message size of 10 KiB; k is the ratio of "bad dpfs" and "bad mailboxes"; y-axis is log-scaled

than Riposte. Due to the logarithmic scaling of the 2-verifier-Sabre-BB's audit protocol, for smaller DPF sizes, the performance difference between the 2-verifier-Sabre-BB and Riposte is not as much as it is for larger DPF sizes. Notice that, when the number of items is large ($\approx 2^{18}$), the difference between the 3-verifier-Sabre-BB and 2-Party-Sabre-BB reduces. As the number of items increases, the time to evaluate the DPFs becomes the bottleneck. As a consequence, the 2-Server and the 3-Server variants show similar performance results for large DPF sizes. The performance improvement of Sabre-BB in comparison with Riposte comes due to (i) a faster audit protocol, and (ii) use of $(\log n)$-sized DPFs (as opposed to Riposte's $\sqrt{n}$-sized DPFs).

7 RELATED WORK

In this section, we very briefly survey various SAM services. Atom [17] is an anonymous micro-blogging application due to Kwon, Corrigan-Gibbs, Devads, and Ford. Atom works in the "bulletin board" model and is most suitable to broadcast short messages. The paper reports that Atom can transit that million Tweet-length messages in 28 minutes. Atom uses public key cryptography to achieve anonymous broadcasting. Cheng, Scott, Masserova, Zhang, Goyal, Anderson, Krishnamurthy, and Prano present Talek [6], a private group messaging system. Talek works in the "mailbox" model. Talek achieves its goals by using information-theoretic PIR. Kwon, Lu and Devadas present XRD [19], a private messaging system providing cryptographic privacy. XRD uses multiple mix networks in parallel. XRD system is designed to work in the "mailbox" model. Kwon, Lazar, Devadas, and Ford present Riffle [18]. Client forms an anonymity set. Servers facilitate the anonymous communication. Clients upload their encrypted messages to the servers. If a client has no message to send, then he/she sends a random string. The servers share all ciphertexts with each other. PIR is used to retrieve messages. Angel and Setty present Pung [2] is a single server system which has inexpensive writes; however the reads are more expensive. Pung uses Computational Private Information Retrieval (CPIR), which allows Pung to provide private communication even if all the servers are malicious. Riposte [9] is the state-of-the-art sender anonymous messaging system in the "bulletin board" model. Clients using Riposte use $O(\sqrt{n})$-sized DPFs to do the writes. The servers use a 3-server audit protocol to check that DPFs being sent are well-formed. Express [11] is the state-of-the-art sender anonymous messaging system in the "mailbox" model. The clients who wan to receive messages via Express register with a mailbox. Express, unlike Riposte use $O(\lg n)$-sized DPFs. Express server expand the DPFs and

the run an audit protocol to ensure that the expansion of the DPFs are indeed a standard basis vector. Express unlike Riposte is a 2-Server audit protocol. There have been other systems like Vuvuzela [23], Stadium [22] and Karaoke [20] which provide differentially private messaging.

8 CONCLUSION

In this paper, we presented Sabre, a sender-anonymous messaging system that works in both the "mailbox model" (Sabre-M) and the "bulletin model" (Sabre-BB). Sabre improves upon Express and Riposte, which are the state-of-the-art in the "mailbox model" and the "bulletin board", respectively. Sabre takes a more holistic view of scalability. In particular, Sabre improves the rate at which the "good" queries are processed and improves the rate at which the "bad" queries are rejected. Quickly rejecting "bad" queries provides Sabre with reliance towards Denial-of-Service attacks. The main innovations of Sabre are: (i) Introducing a "multi-verifier" model to do MPC-in-the-head proofs, (ii) Replacing the server–side audit protocol with a client-side Zero-Knowledge Proof, and (iii) Using PRF to generate the mailbox's virtual addresses.

REFERENCES

[1] Martin R. Albrecht, Christian Rechberger, Thomas Schneider, Tyge Tiessen, and Michael Zohner. Ciphers for MPC and FHE. In *Advances in Cryptology: Proceedings of EUROCRYPT 2015 (Part I)*, volume 9056 of *LNCS*, pages 430–454, Sofia, Bulgaria (April 2015).

[2] Sebastian Angel and Srinath T. V. Setty. Unobservable communication over fully untrusted infrastructure. In *Proceedings of OSDI 2016*, pages 551–569, Savannah, GA, USA (November 2016).

[3] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing. In *Advances in Cryptology: Proceedings of EUROCRYPT 2015 (Part II)*, volume 9057 of *LNCS*, pages 337–367, Sofia, Bulgaria (April 2015).

[4] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing: Improvements and extensions. In *Proceedings of CCS 2016*, pages 1292–1303, Vienna, Austria (October 2016).

[5] Melissa Chase, David Derler, Steven Goldfeder, Claudio Orlandi, Sebastian Ramacher, Christian Rechberger, Daniel Slamanig, and Greg Zaverucha. Post-quantum zero-knowledge and signatures from symmetric-key primitives. In *Proceedings of CCS 2017*, pages 1825–1842, Dallas, TX, USA (October 2017).

[6] Raymond Cheng, William Scott, Elisaweta Masserova, Irene Zhang, Vipul Goyal, Thomas E. Anderson, Arvind Krishnamurthy, and Bryan Parno. Talek: Private group messaging with hidden access patterns. In *Proceedings of ACSAC 2020*, pages 84–99, Austin, TX, USA (December 2020).

[7] Benny Chor, Eyal Kushilevitz, Oded Goldreich, and Madhu Sudan. Private information retrieval. *Journal of the ACM (JACM)*, 45(6):965–981, November 1998.

[8] Henry Corrigan-Gibbs and Dan Boneh. Prio: Private, robust, and scalable computation of aggregate statistics. In *Proceedings of USENIX Security 2017*, pages 259–282, Boston, MA, USA (March 2017).

[9] Henry Corrigan-Gibbs, Dan Boneh, and David Mazières. Riposte: An anonymous messaging system handling millions of users. In *Proceedings of IEEE S&P 2015*, pages 321–338, San Jose, CA, USA (May 2015).

[10] Wenliang Du and Mikhail J. Atallah. Protocols for secure remote database access with approximate matching. In *E-Commerce Security and Privacy (Part II)*, volume 2 of *Advances in Information Security*, pages 87–111, February 2001.

[11] Saba Eskandarian, Henry Corrigan-Gibbs, Matei Zaharia, and Dan Boneh. Express: Lowering the cost of metadata-hiding communication with cryptographic privacy. In *Proceedings of USENIX Security 2021*, Vancouver, BC, Canada (August 2021).

[12] Irene Giacomelli, Jesper Madsen, and Claudio Orlandi. ZKBoo: Faster zero-knowledge for boolean circuits. In *Proceedings of USENIX Security 2016*, pages 1069–1083, Austin, TX, USA (August 2016).

[13] Niv Gilboa and Yuval Ishai. Distributed point functions and their applications. In *Advances in Cryptology: Proceedings of EUROCRYPT 2014*, volume 8441 of *LNCS*, pages 640–658, Copenhagen, Denmark (May 2014).

[14] Ryan Henry and Adithya Vadapalli. dpf++; version 0.0.1 [computer software]. Available from: https://www.github.com/rh3nry/dpfplusplus, July 2019.

[15] Yuval Ishai, Eyal Kushilevitz, Rafail Ostrovsky, and Amit Sahai. Zero-knowledge from secure multiparty computation. In *Proceedings of STOC 2007*, pages 21–30, San Diego, CA, USA (June 2007).

[16] Jonathan Katz, Vladimir Kolesnikov, and Xiao Wang. Improved non-interactive zero knowledge with applications to post-quantum signatures. In *Proceedings of CCS 2018*, pages 525–537, Toronto, ON, Canada (October 2018).

[17] Albert Kwon, Henry Corrigan-Gibbs, Srinivas Devadas, and Bryan Ford. Atom: Horizontally scaling strong anonymity. In *Proceedings of SOSP 2017*, pages 406–422, Shanghai, China (October 2017).

[18] Albert Kwon, David Lazar, Srinivas Devadas, and Bryan Ford. Riffle: An efficient communication system with strong anonymity. *Proceedings on Privacy Enhancing Technologies (PoPETS)*, 2016(2):115–134, April 2016.

[19] Albert Kwon, David Lu, and Srinivas Devadas. XRD: Scalable messaging system with cryptographic privacy. In *Proceedings of NSDI 2020*, pages 759–776, Santa Clara, CA, USA (February 2020).

[20] David Lazar, Yossi Gilad, and Nickolai Zeldovich. Karaoke: Distributed private messaging immune to passive traffic analysis. In *Proceedings of OSDI 2018*, pages 711–725, Carlsbad, CA, USA (October 2018).

[21] S. M. Matyas, C. H. Meyer, and J. Oseas. Generating strong one-way functions with cryptographic algorithm. *IBM Technical Disclosure Bulletin*, 27(10A):5658–565, March 1985.

[22] Nirvan Tyagi, Yossi Gilad, Derek Leung, Matei Zaharia, and Nickolai Zeldovich. Stadium: A distributed metadata-private messaging system. In *Proceedings of SOSP 2017*, pages 423–440, Shanghai, China (October 2017).

[23] Jelle van den Hooff, David Lazar, Matei Zaharia, and Nickolai Zeldovich. Vuvuzela: Scalable private messaging resistant to traffic analysis. In *Proceedings of SOSP 2015*, Monterey, CA, USA (October 2015).

A PROOFS

A.1 Proof of Observation 1

Proof. The proof that all leaves of a tree rooted at a 0-node are of type 0 is trivial; the proof that a tree rooted at a 1-node has exactly one type-1 leaf with all others being of type 0 is by induction on the height h of the tree. The base case $h = 0$ follows immediately from the definition of a 1-node. For the inductive step, suppose that every height-h tree with a type-1 root has exactly one type-1 leaf with all

other leaves being of type 0. Given a height-$(h+1)$ tree with a type-1 root, there are two possibilities: either the leftmost subtree is rooted at a 1-node and the rightmost subtree is rooted at a 0-node, or vice versa. In both cases, one subtree has only type-0 leaves, while, by the inductive hypothesis, the other subtree has a single type-1 leaf with all others being of type 0. □

B DU-ATALLAH MULTIPLICATION

Du-Atallah multiplication is a method of doing Beaver multiplication while avoiding the expensive OT (which exists in the beaver multiplication) at the expense of introducing a third party.

Suppose that $P0$ has $([\![x]\!]_0, [\![x]\!]_1)$ and $P1$ has $([\![y]\!]_0, [\![y]\!]_1)$. $P2$ samples $(X_0, Y_0, X_1, Y_1, \alpha)$. Then, $P2$ sends $(X_0, Y_0, X_0Y_1+\alpha)$ to $P0$ and $(X_1, Y_1, X_1Y_0-\alpha)$. $P1$ then sends $([\![x]\!]_1+X_1)$ and $([\![y]\!]_1+Y_1)$ to $P0$ and $P0$ sends $([\![x]\!]_0+X_0)$ and $([\![y]\!]_0+Y_0)$.

$P0$ then computes,

$$[\![z]\!]_0 := [\![x]\!]_0 \cdot ([\![y]\!]_0 + ([\![y]\!]_1 + Y_1)) - Y_0 \cdot ([\![x]\!]_1 + X_1) + (X_0Y_1 + \alpha)$$

Similarly, $P1$ computes,

$$[\![z]\!]_1 := [\![x]\!]_1 \cdot ([\![y]\!]_1 + ([\![y]\!]_0 + Y_0)) - Y_1 \cdot ([\![x]\!]_0 + X_0) + (X_1Y_0 + \alpha)$$

We then note that,

$$[\![z]\!]_0 + [\![z]\!]_1 = (X_0 + X_1) \cdot (Y_0 + Y_1)$$

C FORMALIZATION OF "MULTI-VERIFIER" MPC-IN-THE-HEAD

SNIPs. Let $[w]$ be a secret shared among n shareholders using some (t, n)-threshold secret sharing scheme and let $R(x, w)$ be an NP-relation. Here x is a public input and w is the witness. A *secret-shared non-interactive proof (SNIP)* is a protocol that allows a client to prove their knowledge of w to the n shareholders.

Let $R(x, w)$ be an NP-relation, where x is a public input and w is the witness. The prover wishes to prove the knowledge of the witness w. SNIPs work in a setting where the verifiers are the shareholders of the witness. In other words, the verifiers (shareholders) hold $[w]$ and the goal of the client (prover) who holds w, is to convince the verifiers that $R(x, w) = 1$.

(1) **Correctness:** If all parties are honest, the servers will accept x.
(2) **Soundness:** If $R(x, w) \neq 1$, then the servers accept the proof with negligible probability.
(3) **Zero knowledge:** If the client and at least one shareholder are honest, then the shareholders learn nothing about w, except that $R(x, w) = 1$.

"Multi-Verifier" MPC-in-the-head. In this section, we show the MPC-in-the-head paradigm with multiple verifiers can be used to construct SNIPs. In this section, we will concern ourselves with only, 1-private MPC protocol.

Recall that in MPC-in-the-head, the prover holds the witness w, and shares it among n imaginary shareholders. A subtle but important difference between SNIPs and MPC-in-the-head is that in SNIPs the protocol begins with the shareholders holding the shares, but in MPC-in-the-head, the prover shares the witness w among the shareholders. Therefore, if we are having multiple verifies in the MPC-in-the-head setting, the fact that the verifiers do not have any private inputs means that there are no restrictions on the set of transcripts each verifier could check. However, in a SNIP, one needs to ensure that the verifiers cannot combine the share they hold with the views of the simulated parties to generate the secret.

Assume that, $\mathcal{V}$ is a set of r verifiers, namely, $V_1, \ldots, V_r$. A subset $\mathcal{S} \subseteq \mathcal{V}$, namely, $V_1, \ldots, V_{|S|}$ hold private inputs, $w_1, \ldots, w_{|S|}$, respectively. The prover's goal is to convince the verifiers that, $f(w_1, w_2, \ldots, w_{|S|}) = 1$.

The prover in their head, runs a 1-private, n-party protocol, to evaluate $f(w_1, w_2, \ldots, w_{|S|})$ (where $n \geq r$). Suppose that, $P0$, ..., Pn, are the parties involved in the MPC protocol. Also, w.l.o.g, that, $P0, \ldots, P|S|$, hold private inputs, $(w_1, \ldots, w_{|S|})$.

The verification process, has the following restriction that Verifiers $V_1, \ldots, V_{|S|}$ can only verify transcripts of parties $P0, \ldots, P|S|$. The remaining verifiers do not have any restrictions on the transcripts they can verify.

The proof is accepted if and only if, for all $j \in \{1, \ldots, r\}$:

- V_j finds that the views of Pj to be consistent.
- V_j finds that the outputs of all the party Pj to be 1.

Lemma 1. *(Soundness): If all the verifiers accept the proof then, the probability that the relation doesn't hold* $1 - r/n$.

Definition 5. *(Completeness): If* $(x, w) \in R$*, then all the views have output 1. Since the views are valid, they are consistent with each other.*

Definition 6. *(Zero-Knowledge): There exists a simulator that can simulate the views of* t *of the verifiers.*

Observation 2. *If* $r = n$*, i.e. the number of verifiers is equal to the number of parties in the MPC protocol (being run in the head), we get perfect soundness.*

D THREE VERIFIER EXTENSION

In this section, we show how the 2-server verification can be extended to 3-server verification. The price of using an extra server nearly gives us ≈ 128 times speed up in the verification and proof construction process.

D.1 3-Server ZKP

A natural extension to the 2-Server Sabre is to add a third verifier. Then, each server checks the transcript from the simulated party, which was given the same DPF key as that server. The third server, which does not receive a DPF key,

verifies the third party's transcript in the simulated protocol, which does not receive a DPF key. Each party in the simulated MPC protocol has its transcript verified by one of the servers. As a result, the MPC protocol's correctness causes any fake proof to contain an inconsistency in at least one party's transcript [15]. However, since all of the parties have had their transcript checked, at least one of the verifiers will detect invalid DPF proof. Having three parties means that there is no need to perform multiple repetitions as the soundness error is 0. The zero-knowledge property of the ZKP is still satisfied because the three servers are assumed to be non-colluding. It also allows verifying K proofs in parallel.

The proof construction in 3-server Sabre is the same as the proof construction in 2-server Sabre. The messages sent by the prover to the servers is different in the 3-server setting. The prover constructs the merkle-tree (call the root as $\mathcal{R}$). Prover after constructing the merkle-tree (like in 2-server Sabre) sends the following messages.

(1) $(\mathcal{R}, \mathsf{seed}_i, \mathcal{H}_i[0,1], \mathcal{H}_i[1,0])$ to $P2$.
(2) $(\mathcal{R}, \mathbf{m}_i[0,1], \mathbf{m}_i[2,1], \mathcal{H}_i[2,0])$ to $P1$.
(3) $(\mathcal{R}, \mathbf{m}_i[1,0], \mathbf{m}_i[2,0], \mathcal{H}_i[2,1])$ to $P0$.

In the verification process, $P0$, $P1$ and $P2$ reconstruct the root $\mathcal{R}'$ and check if $\mathcal{R}$ equals $\mathcal{R}'$. If all the three parties find that $\mathcal{R} = \mathcal{R}'$, the proof is accepted. The 3-server verification, denoted as $\mathsf{verification}_i^{3P}$, $i \in \{0,1,2\}$ is elaborated as follows.

(1) $\mathsf{verification}_2^{3P}$: $P2$ uses seed_i to generate $\mathbf{m}_i[2,1]'$ and $\mathbf{m}_i[2,0]'$.
(2) $\mathsf{verification}_1^{3P}$: $P1$ use $\mathbf{m}_i[0,1]$ and $\mathbf{m}_i[2,1]$ to construct $\mathbf{m}_i[0,1]'$.
(3) $\mathsf{verification}_0^{3P}$: $P0$ use $\mathbf{m}_i[1,0]$ and $\mathbf{m}_i[2,0]$ to construct $\mathbf{m}_i[1,0]'$.

$P2$, $P1$ and $P0$, after this verification have all the messages required to reconstruct the root of the merkle-tree root. If the reconstructed merkle-tree root is denoted as $\mathcal{R}'$, then $P0$, $P1$ and $P2$ verify if $\mathcal{R}$ (the merkle-tree root send by the prover) is equal to $\mathcal{R}'$ (the merkle-tree root reconstructed by the verifier). The proof is accepted if all the parties find that $\mathcal{R}$ and $\mathcal{R}'$ are equal.

E COMPLETENESS AND SOUNDNESS PROOFS

E.1 2-Server

Completeness. To show the completeness, we need to show that if the prover is honest, the verifier will accept the proof with probability 1. The completeness essentially follows from the construction. If the prover is honest, (i) for $c_i = 0$, Pb, an honest prover would send the correct seed_i; thus $\mathbf{m}_i[2,1]' = \mathbf{m}_i[2,1]$ and $\mathbf{m}_i[2,0]' = \mathbf{m}_i[2,0]$, and (ii) for $c_i = 1$, an honest prover would send the correct Pb can generate the correct $\mathbf{m}_i[b, 1-b]$ and $\mathbf{m}_i[2,b]$. Therefore, $\mathbf{m}_i[1-b,b]' = \mathbf{m}_i[1-b,b]$. Thus, $\mathcal{R}'$ would be equal to $\mathcal{R}$ and hence the verifiers would accept the proof with probability 1.

Soundness. For every iteration, the verifiers either verify that the randomness generated by $P2$ is correct or given that the randomness generated by $P2$ is correct, the interaction between $P0$ and $P1$ is correct. The probability of succeeding for a cheating prover is $1/2$ per iteration. Since verification is repeated K times, the probability of cheating is $1/2^K$.

E.2 3-Server

Completeness. Completeness of the 3-Verifier protocol follows from the fact if prover is honest, then $\mathbf{m}_i[2,0]' = \mathbf{m}_i[2,0]$, $\mathbf{m}_i[2,1]' = \mathbf{m}_i[2,1]$, $\mathbf{m}_i[1,0]' = \mathbf{m}_i[1,0]$ and $\mathbf{m}_i[0,1]' = \mathbf{m}_i[0,1]$.

Soundness. In the verifier setting, all the transcripts are verified by *at least* one verifier. Therefore, the probability of cheating is 0 and hence the soundness error for 3-Verifier setting is 0.

F ZERO-KNOWLEDGE PROOF IMPLEMENTATION OPTIMIZATIONS

We now present the various optimizations in our implementation.

Transposed Implementation. In order to make use of the AVX instructions and do the K instances in parallel, we transpose our inputs. Throughout this description, assume that $K \in \{128, 256\}$ (as that would allow us to use fast SIMD instructions). By transposing and storing the K DPF keys as arrays of m128i or m256i.

- The inputs to the simulator are two 128 bit DPF keys dpf_0 and dpf_1, respectively.
- At the first step, the simulator does a transposition of the two keys into a $128 \cdot K$ matrix, $\vec{K}_0$ and $\vec{K}_1$ respectively, such that $\vec{K}_b[i,t] = \mathsf{dpf}_b[t]$, for all i. In other words, the ith row of $\vec{K}_b$ is an all ones row if the ith bit of $\mathsf{dpf}_b = 1$, otherwise it an all zeros column.
- It is clear that the ith column of the matrix corresponds to the ith parallel run of the protocol.

The key idea behind transposition is that we would be doing the computation for each level of the K iterations in parallel.

Our modified challenge. Instead of choosing a random 128-bit challenge, we choose a random challenge with *exactly* 64 bits set (32 of the bits are set among the low 64 bits, and 32 are set among the high 64 bits). The root of the Merkle-tree is used to construct the challenge. This ensures that the size of every message sent among $P0$ and $P1$ is a multiple of 64, thus allowing us to pack everything in 64-bit integers. Without this restricted challenge, there would be no way to efficiently

pack an arbitrary number of bits. Since there are $\binom{64}{32}$ bit numbers with exactly 32 bits set, this restricted challenge only reduces the soundness by only 7 bits.

G SABRE COMMUNICATION COST

We first calculate the size of $\mathbf{m}_i(b, 1-b)$. The transcript from $P0$ to $P1$ include the following (the transcript of $P1$ to $P0$ is the same).

(1) shares of the children of the root = 16 bytes,
(2) oblivious encryption, $\mathcal{F}_{\mathsf{PRG}} = (4 \cdot 3 \cdot \mathsf{s} \cdot \mathsf{r}/8)(h-1)$ bytes,
(3) conditional swap, $\mathcal{F}_{\mathsf{SWAP}} = 16 \cdot h$ bytes,
(4) applying the correction word $= 16 \cdot (h-1)$ bytes,
(5) computing the next flag bit $(1/8 + 1/8) \cdot (h-1)$ bytes.

size of $\mathbf{m}_i(b, 1-b) = 16 + (16\frac{1}{8} + \frac{3}{2}\,\mathsf{s}\,\mathsf{r})(h-1)$ Next, we calculate the size $M_i(2, b)$ for $b \in \{0, 1\}$.

(1) seed to generate the randomness = 16 bytes,
(2) oblivious encryption, $\mathcal{F}_{\mathsf{PRG}} = (4 \cdot 3 \cdot \mathsf{s} \cdot \mathsf{r}/8)(h-1)$,
(3) applying the correction word $= 16 \cdot (h-1)$ bytes,
(4) conditional swap, $\mathcal{F}_{\mathsf{SWAP}} = 16 \cdot h$ bytes,
(5) computing the next flag bit $(1/8) \cdot (h-1)$ bytes.

size of $\mathbf{m}_i(b, 1-b) = 16 + (32\frac{1}{8} + \frac{3}{2}\,\mathsf{s}\,\mathsf{r})(h-1)$

G.1 2-Verifier Sabre

Client to Pb. In the 2-Verifier Sabre, the client sends the following to Pb, for $b \in \{0, 1\}$. (i) DPF seed, and (ii) Proof of the well-formedness of the DPF.

Proof includes the following:

(A) Merkle Tree Root = (16 bytes)
(B) for 64 iterations:
- $\mathbf{m}_i(b, 1-b)$,
- $\mathbf{m}_i(2, b)$,
- $\mathcal{H}(\mathbf{m}_i(2, 1-b))$, $\mathcal{H}(\mathbf{m}_i(2, b)) = (2 \cdot 16 = 32)$ bytes.

(C) for 64 iterations:
- a seed $= (64 \cdot 16)$ bytes,
- $\mathcal{H}(\mathbf{m}_i(b, 1-b))$, $\mathcal{H}(\mathbf{m}_i(1-b, b)) = (2 \cdot 16 = 32)$ bytes.

G.2 3-Verifier Sabre

In a 3-Verifier Sabre, the client sends the following messages to $P2$.

(A) Merkle Tree Root = (16 bytes),
(B) $\mathcal{H}(\mathbf{m}(b, 1-b)), \mathcal{H}(\mathbf{m}(1-b, b)) = (2 \cdot 16$ bytes),
(C) seed = 16 bytes.

Similarly, the client sends the following message to Pb ($b \in \{0, 1\}$).

(A) Merkle Tree Root = (16 bytes),
(B) $\mathbf{m}(b, 1-b)), \mathbf{m}(1-b, b)$,
(C) $\mathcal{H}(\mathbf{m}(2, b))) = (16$ bytes).

H VERIFYING MAILBOX ADDRESS

Sabre-M1 and Sabre-M2 use PIR-based mailbox verification. Sabre-M, on the other hand, uses PRF-based verification.

H.1 PIR based verification

PIR based verification is used Sabre-M1 and Sabre-M2. In these two versions, the mailbox addresses are generated exactly like in Express (and in Sabre-M0). The servers maintain a database, ADDR, mapping the virtual address to the physical address of the mailbox. The prover wants to prove that they know the physical address corresponding to the virtual address (the virtual address is the underlying index of the DPF). The client along with DPF keys submits $(2, 2)$-additive share $addr_b \in \mathbb{F}_{2^\lambda}$ of the λ-bit mailbox address $addr$ to server b for $b = 0, 1$. The servers then use the DPFs, dpf_0 and dpf_1 to do a PIR query into the address database ADDR to get $addr'_0$ and $addr'_1$. The servers then verify that $(addr_0 + addr'_0 + addr_1 + addr'_1) = 0$. The PIR-based verification takes $\mathrm{O}(n)$ time.

H.2 PRF based verification

Sabre-M uses the PRF based verification. The mailbox addresses in Sabre-M are generated using a *secret* LowMC key, prgkey. In particular, the ith mailbox's address is LowMC encryption of i with prgkey.

Like in Sabre-M1 and Sabre-M2, the clients in Sabre-M send the shares of the physical mailbox addresses, $addr_0$ and $addr_1$ to servers 0 and 1 respectively. The servers then do an MPC-based computation of the shares of the mailbox addresses $addr'_0$ and $addr'_1$ respectively. Post that, the servers then verify that $(addr_0 + addr'_0 + addr_1 + addr'_1) = 0$.

I LOWMC

The PRG used in this protocol is built using black-box access to a pseudorandom generator (PRG). For our implementation, the LowMC block cipher [1] is used to construct the PRG. LowMC is a block cipher designed for use in secure multiparty computation, fully homomorphic encryption, and zero-knowledge proofs. In these applications, non-linear operations have an extremely high cost, and linear operations have almost no cost. To minimize these costs, LowMC is designed to have low multiplicative complexity and a low multiplicative depth. One method by which this is achieved is the use of a partial s-box layer in which the s-box is only applied to some of the bits, and the remaining bits are left unchanged.

LowMC is also designed with a wide range of parameterization choices. In particular, for a given number of s-boxes, the number of rounds necessary to achieve the desired security level can be calculated [1]. Using more sboxes decreases the number of rounds, but it increases the number of multiplications that must be performed. In traditional block cipher applications, all of the multiplications necessary for the sboxes

in a round can be implemented using SIMD instructions. In the MPC setting, it is best to minimize the number of rounds because each round of the protocol requires communication between parties, whereas increasing the number of sboxes per round has very little impact on the overall efficiency. However, when used for an MPC-in-the-head ZKP, the MPC protocol is made non-interactive using the Fiat-Shamir transform. This means that the number of rounds used by the protocol is much less significant since communication delay is no longer a consideration. The number of rounds required corresponding to different number of s-boxes is presented in the Appendix in Figure 8.

In an MPC-in-the-head setting, there are two important considerations when choosing the number of sboxes for LowMC. The first consideration is the transcript size. Each sbox used per round will add four bits to the total transcript size. This leads to an increase in the proof size by one or two bits, depending on which randomly selected party in the protocol is being verified by the verifier. However, the second consideration is the total time cost for generating and verifying the proof. Reducing this cost is achieved by minimizing the number of rounds. This is because each round requires the verifier to perform an expensive MPC multiplication. LowMC's highly parameterizable design and its aim of minimizing multiplicative complexity and multiplicative depth make it an excellent choice for use in MPC-in-the-head paradigm ZKPs. The parameters of the LowMC can be adjusted depending on whether we need smaller transcripts or we need faster writes. We present experiments on how to choose the parameters in Appendix J.

I.1 Faster Matrix Multiplication

We also improve upon the performance of LowMC, by implementing a faster matrix multiplication. Our implementation takes advantage of the fact that fixed-keys implies fixed round constants and linear matrices in LowMC. However, because our bit-sliced implementation effectively transposes the "messages" we must also transpose the matrices, which prevents us from using the obvious precomputation-based multiplication with fixed Lookup Tables (LUTs). Instead, our implementation uses a greedy algorithm that pre-computes efficient "recipes" for constructing LUTs using a small number of operations at run-time, shaving about 60% off the cost relative to the naive precomputation-based approach.

J CHOICE OF LOWMC PARAMETERS

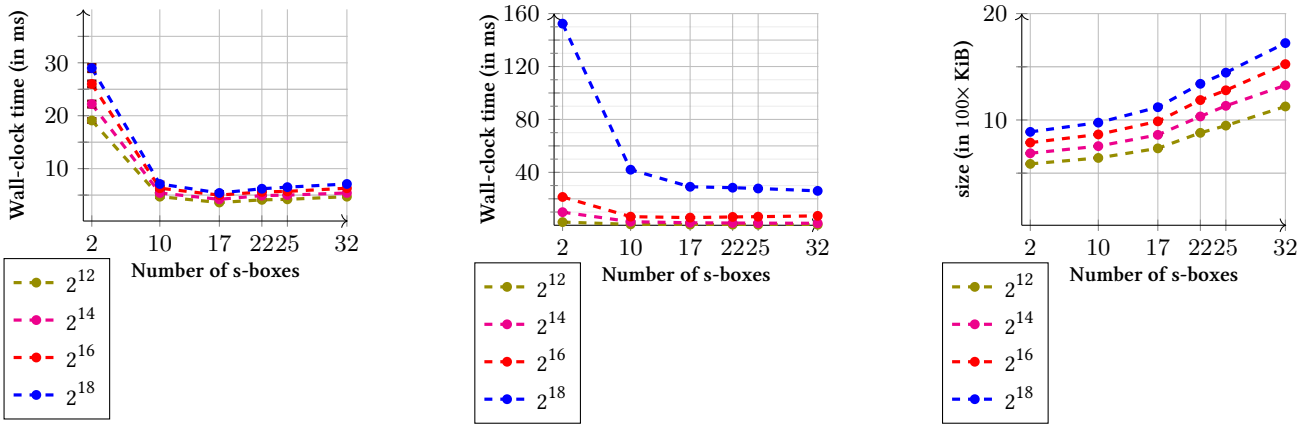
In this section, we present experiments that justify the choice of our LowMC parameters. LowMC allows us to choose different parameters resulting in different computation times and proof sizes. Figure 9 present experiments by varying the number of s-boxes. Figure 9c shows that proof size monotonically increases with the increase in the number of s-boxes. We see from Figure 9a that when the number of s-boxes is 17, the audit-time is optimal. Figure 9b indicates that there is a monotonic decrease in the time to do an Evalfull as the number of s-boxes increase (consequently the number of rounds decrease). It is also worth noting that, while the optimal parameters for the audit time and the Evalfull time are different, the time to do the Evalfull is the dominating part of the cost. Setting the s-boxes to 32 (and thus the number of rounds to 19; See Figure 8 for more details) gives the optimal performance in terms of time to do the write.

We also remark here that when Sabre is under DoS attack, it will skip the evaluation of the DPF. In other words, when there is a large percentage of "bad" queries, the optimal choice of parameters would be when the number of s-boxes is 17. It is noticeable that proof size and the time to do the write (sum of the audit time and the Evalfull time) follows an inverse relation. If the system requirement is to reduce the proof size and the client communication cost, one can choose some other LowMC parameters.

The goal of this paper is to show that Sabre can do faster writes than the state-of-the-art. Since the fastest writes are achieved by setting the number of s-boxes to 32, rounds to 19, and block size to 128, we perform all the experiments in those LowMC parameters. These LowMC parameters do result in relatively larger proof sizes.

Sboxes	Rounds	Data Complexity	Key Size	Block Size
1	287	128	128	128
2	144	128	128	128
3	96	128	128	128
4	72	128	128	128
5	57	128	128	128
6	49	128	128	128
7	42	128	128	128
8	38	128	128	128
9	34	128	128	128
10	32	128	128	128
11	29	128	128	128
12	28	128	128	128
13	27	128	128	128
14	26	128	128	128
15	24	128	128	128
16	23	128	128	128
17	22	128	128	128
22	21	128	128	128
25	20	128	128	128
32	19	128	128	128

Figure 8: LowMC parameters



(a) Audit time vs. number of items

(b) **Evalfull** time vs. number of items

(c) Proof size vs. number of items

Figure 9: Plotting Audit time, Evalfull time and Proof Size versus number of s-boxes for different DPF sizes.